



**ҚР ОҚУ-АҒАРТУ МИНИСТРЛІГІ
МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РК**

**АЛМАТЫ ҚАРЖЫ-ЭКОНОМИКАЛЫҚ КОЛЛЕДЖІ
АЛМАТИНСКИЙ ФИНАНСОВО-ЭКОНОМИЧЕСКИЙ
КОЛЛЕДЖ**

«Қорғауға рұқсат»
Колледж директоры
_____ Алимова К.У.
« ____ » _____ 2026 ж.
хаттама № _____

ДИПЛОМДЫҚ ЖҰМЫС

Тақырыбы «Қайырымдылық жобаларын басқаруға арналған веб-платформа»

Мамандық

06130100 Бағдарламалық қамтамасыз ету
(түрлері бойынша)

Біліктілік

4S06130105 Ақпараттық жүйелер технигі

Орындады:	_____	Нұржанұлы С.
Ғылыми жетекші:	_____	Тұрғантай А.Ә.

Алматы, 2026

АНДАТПА

Бұл дипломдық жұмыста қайырымдылық жобаларын басқаруға арналған веб-платформаны әзірлеудің теориялық, жобалық және тәжірибелік аспектілері қарастырылды. Жұмыста әлеуметтік бағыттағы ақпараттық жүйелердің маңызы, қайырымдылық процестерін цифрландыру ерекшеліктері, веб-қосымшаға қойылатын функционалдық және функционалдық емес талаптар талданды. Практикалық бөлімде Django фреймворкі, SQLite дерекқоры, Bootstrap интерфейсі және әкімші панелі негізінде көп бетті платформа жасалды. Жүйеде жобалар каталогы, көмек сұрау формасы, үлес қосу механизмі, жоба жаңалықтары, аналитикалық dashboard және қазақ тілді admin панель іске асырылды.

АННОТАЦИЯ

В данной дипломной работе рассмотрены теоретические, проектные и практические аспекты разработки веб-платформы для управления благотворительными проектами. Проанализированы значение цифровизации благотворительных процессов, требования к социальной информационной системе, структура базы данных и особенности пользовательского интерфейса. Практическая часть выполнена на основе Django, SQLite и Bootstrap. В системе реализованы каталог проектов, форма заявки на помощь, механизм предложений помощи, новости проектов, аналитическая панель и административная панель на казахском языке.

ABSTRACT

This diploma thesis examines the theoretical, design and practical aspects of developing a web platform for managing charity projects. The work analyzes the importance of digitalizing charity processes, requirements for a social information system, database structure and user interface organization. The practical part is implemented using Django, SQLite and Bootstrap. The platform includes a project catalog, help request form, contribution workflow, project updates, analytics dashboard and Kazakh-language administration panel.

МАЗМҰНЫ

КІРІСПЕ.....	4
1 ҚАЙЫРЫМДЫЛЫҚ ВЕБ-ПЛАТФОРМАСЫНЫҢ ТЕОРИЯЛЫҚ НЕГІЗДЕРІ.....	6
1.1 Қайырымдылық қызметін цифрландырудың маңызы.....	6
1.2 Қайырымдылық жобаларын басқару ерекшеліктері.....	7
1.3 Веб-платформаға қойылатын негізгі талаптар.....	8
2 ВЕБ-ПЛАТФОРМАНЫ ЖОБАЛАУ.....	10
2.1 Жүйенің мақсаты, міндеттері және пайдаланушы рөлдері.....	10
2.2 Деректер қорының құрылымын жобалау.....	12
2.3 Жүйе архитектурасы және интерфейс құрылымы.....	14
3 ВЕБ-ПЛАТФОРМАНЫ ІСКЕ АСЫРУ.....	17
3.1 Django жобасының құрылымы және негізгі модульдері.....	17
3.2 Негізгі функционалдарды әзірлеу.....	20
3.3 Тестілеу және демо деректермен толтыру.....	26
4 ЕҢБЕКТІ ҚОРҒАУ.....	29
4.1 Компьютермен жұмыс істеу қауіпсіздігі.....	29
4.2 Жұмыс орнын ұйымдастыру талаптары.....	30
4.3 Ақпараттық қауіпсіздік және деректерді қорғау.....	32
5 ЭКОНОМИКАЛЫҚ БӨЛІМ.....	34
5.1 Жобаның экономикалық және әлеуметтік мәні.....	34
5.2 Жобаны әзірлеу шығындарын есептеу.....	35
5.3 Жобаның тиімділігі және даму мүмкіндіктері.....	37
ҚОРЫТЫНДЫ.....	39
ПАЙДАЛАНЫЛҒАН ӘДЕБИЕТТЕР.....	40
ҚОСЫМШАЛА А.....	41

KIPICPE

Қазіргі қоғамда ақпараттық технологиялар адам өмірінің барлық саласына терең еніп, әлеуметтік, экономикалық және мәдени процестерді жаңа деңгейге көтеріп отыр. Соның ішінде қайырымдылық қызметін ұйымдастыру, көмекке мұқтаж азаматтарды қолдау, еріктілер мен демеушілердің жұмысын үйлестіру сияқты бағыттар да цифрлық шешімдерді қажет етеді. Бұрын қайырымдылық шаралары көбіне әлеуметтік желілер, мессенджерлер немесе жеке байланыстар арқылы ұйымдастырылса, қазіргі кезде мұндай процестерді бір орталықтан басқаруға мүмкіндік беретін веб-платформалардың маңызы артып келеді.

Қайырымдылық жобаларын басқару барысында көптеген ақпарат өңделеді: көмекке мұқтаж адамдар туралы мәліметтер, жоба сипаттамасы, қажетті қаражат мөлшері, жиналған сома, көмек көрсетушілердің өтініштері, жоба мәртебесі және атқарылған жұмыс туралы жаңалықтар. Егер бұл ақпарат жүйесіз сақталса, деректердің қайталануы, қателіктердің пайда болуы, есеп жүргізудің қиындауы және пайдаланушылардың сенімінің төмендеуі мүмкін. Сондықтан қайырымдылық қызметін автоматтандыру және ашық түрде көрсету – қазіргі заман талабына сай өзекті мәселе.

Осы дипломдық жұмыстың тақырыбы – «Қайырымдылық жобаларын басқаруға арналған веб-платформа». Бұл тақырыптың өзектілігі көмек сұраушылар, демеушілер, еріктілер және модераторлар арасындағы байланысты жеңілдету қажеттілігімен түсіндіріледі. Веб-платформа арқылы қайырымдылық жобаларын жариялау, оларды категориялар бойынша бөлу, жиналған қаражатты бақылау, көмекке өтініш қабылдау және үлес қосу ұсыныстарын тіркеу мүмкін болады. Мұндай жүйе қайырымдылық процесін ашық, түсінікті және тиімді ұйымдастыруға көмектеседі.

Дипломдық жұмыстың мақсаты – қайырымдылық жобаларын басқаруға арналған веб-платформаны жобалау және оның негізгі функционалдық мүмкіндіктерін іске асыру. Аталған мақсатқа жету үшін жүйенің теориялық негіздері қарастырылып, веб-платформаға қойылатын талаптар анықталады, деректер қорының құрылымы жобаланады, пайдаланушы интерфейсі жасалады және негізгі модульдер іске асырылады.

Осы мақсатқа жету үшін келесі міндеттер қойылды:

- қайырымдылық қызметін цифрландырудың маңызын анықтау;
- қайырымдылық жобаларын басқару ерекшеліктерін талдау;
- веб-платформаға қойылатын негізгі талаптарды сипаттау;
- жүйенің пайдаланушы рөлдерін анықтау;
- деректер қорының құрылымын жобалау;
- Django фреймворкі негізінде веб-платформаның негізгі модульдерін іске асыру;
- көмекке өтініш беру және үлес қосу функционалын әзірлеу;
- әкімші панелі мен статистикалық бөлімнің жұмысын ұйымдастыру;

— жүйені демо деректермен толтырып, жұмысын тексеру.

Зерттеу нысаны – қайырымдылық жобаларын басқару және әлеуметтік көмек көрсету процесі. Зерттеу пәні – қайырымдылық жобаларын цифрлық ортада басқаруға арналған веб-платформаның құрылымы мен функционалдық мүмкіндіктері.

Дипломдық жұмыста Django фреймворкі, SQLite деректер қоры, Bootstrap кітапханасы және HTML/CSS технологиялары қолданылды. Бұл құралдар веб-платформаны жылдам әзірлеуге, деректерді жүйелі сақтауға, қолданушыға ыңғайлы интерфейс құруға және әкімші панелі арқылы ақпаратты басқаруға мүмкіндік береді.

Жұмыстың практикалық маңызы – әзірленген веб-платформаны шағын қайырымдылық ұйымдары, еріктілер топтары немесе оқу орындарындағы әлеуметтік жобаларды басқару үшін қолдануға болады. Жүйе арқылы белсенді жобаларды бақылау, аяқталған көмектерді көрсету, өтініштерді тіркеу және үлес қосу ұсыныстарын қабылдау мүмкіндігі қарастырылған.

Дипломдық жұмыс кіріспеден, теориялық бөлімнен, жобалау бөлімінен, практикалық іске асыру бөлімінен, еңбекті қорғау бөлімінен, экономикалық бөлімнен, қорытындыдан, пайдаланылған әдебиеттер тізімінен және қосымшалардан тұрады.

1 ҚАЙЫРЫМДЫЛЫҚ ВЕБ-ПЛАТФОРМАСЫНЫҢ ТЕОРИЯЛЫҚ НЕГІЗДЕРІ

1.1 Қайырымдылық қызметін цифрландырудың маңызы

Қайырымдылық веб-платформасының теориялық негіздерін қарастыру оның әлеуметтік маңызын, ақпараттық жүйе ретіндегі ерекшеліктерін және пайдаланушыға ұсынатын мүмкіндіктерін түсінуге мүмкіндік береді. Қайырымдылық саласында веб-платформаны қолдану тек техникалық шешім ғана емес, сонымен қатар әлеуметтік процестерді ұйымдастырудың тиімді тәсілі болып саналады.

Қайырымдылық жобаларын басқаруға арналған веб-платформа көмекке мұқтаж азаматтар, көмек көрсетушілер, еріктілер және модераторлар арасындағы байланысты бір жүйеге біріктіреді. Мұндай жүйе арқылы жобалар туралы ақпаратты жариялау, қажетті қаражатты көрсету, жиналған соманы бақылау, жоба мәртебесін өзгерту және атқарылған жұмыс бойынша жаңалықтар енгізу мүмкін болады.

Теориялық тұрғыдан алғанда, мұндай платформа бірнеше негізгі бағытқа сүйенеді: әлеуметтік көмек көрсету процесін цифрландыру, деректерді құрылымдау, пайдаланушылардың рөлдерін бөлу, ақпараттың ашықтығын қамтамасыз ету және веб-қосымшаның қауіпсіз жұмысын ұйымдастыру. Осы бөлімде қайырымдылық қызметін цифрландырудың маңызы, қайырымдылық жобаларын басқару ерекшеліктері және веб-платформаға қойылатын негізгі талаптар қарастырылады.

Қайырымдылық қызметі – қоғамдағы әлеуметтік мәселелерді шешуге бағытталған маңызды бағыттардың бірі. Ол көмекке мұқтаж адамдарды қолдау, қиын жағдайда қалған отбасыларға жәрдем беру, денсаулық сақтау, білім беру, тұрғын үй, азық-түлік және басқа да қажеттіліктер бойынша көмек көрсету сияқты көптеген салаларды қамтиды. Қайырымдылық жұмысының тиімді болуы үшін ақпараттың дұрыс жиналуы, тексерілуі, жүйеленуі және көмек көрсетушілерге түсінікті түрде ұсынылуы қажет.

Дәстүрлі жағдайда қайырымдылық жобалары туралы ақпарат әлеуметтік желілерде, мессенджерлерде немесе жеке парақшаларда жарияланады. Бұл тәсіл белгілі бір деңгейде жылдам болғанымен, оның бірнеше кемшілігі бар. Біріншіден, ақпарат әртүрлі жерде шашыраңқы орналасады. Екіншіден, жоба бойынша қанша қаражат жиналғанын, қанша қаражат қалғанын және көмектің қандай кезеңде екенін бақылау қиын болады. Үшіншіден, жоба аяқталғаннан кейін нәтижені жүйелі түрде көрсету әрдайым орындала бермейді.

Цифрландыру осы мәселелерді шешуге мүмкіндік береді. Веб-платформа арқылы барлық қайырымдылық жобаларын бір ортада жинақтап, әр жобаның сипаттамасын, категориясын, қаласын, маңыздылық деңгейін, мақсатты сомасын және жиналған қаражатын көрсетуге болады. Бұл көмек көрсетушілерге жобаны таңдауды жеңілдетеді, ал модераторларға ақпаратты басқаруды оңайлатады.

Қайырымдылық қызметін цифрландырудың тағы бір маңызды артықшылығы – ашықтық. Пайдаланушы жоба бетіне кірген кезде жобаның мақсатын, қажетті сомасын, жиналған қаражатын және прогресс көрсеткішін көре алады. Егер жоба бойынша жаңалықтар жарияланса, демеушілер өз көмегінің қалай қолданылып жатқанын бақылауға мүмкіндік алады. Бұл қайырымдылыққа деген сенімді арттырады.

Сонымен қатар цифрлық платформа уақытты үнемдейді. Мысалы, көмекке өтініш беру үшін адам мекемеге бармай-ақ, арнайы форма арқылы өзінің аты-жөнін, байланыс нөмірін, қаласын, көмек түрін және жағдайын сипаттап жібере алады. Ал модератор бұл өтінішті admin панель арқылы қарап, мәртебесін өзгерте алады. Осылайша өтініш қабылдау және өңдеу процесі жылдамдайды.

Қайырымдылық қызметін цифрландыру еріктілердің жұмысын да жеңілдетеді. Еріктілер белсенді жобаларды көріп, қай бағытта көмек қажет екенін анықтай алады. Бірі қаржылай көмек ұсынса, екіншісі заттай көмек немесе ерікті ретінде қатысу мүмкіндігін таңдай алады. Мұндай құрылым қайырымдылық процесін нақты әрі ұйымдасқан түрде жүргізуге жағдай жасайды.

Осы себептерге байланысты қайырымдылық жобаларын басқаруға арналған веб-платформа әлеуметтік сала үшін маңызды құрал болып табылады. Ол ақпаратты жүйелейді, пайдаланушылар арасындағы байланысты жеңілдетеді, ашықтықты арттырады және көмек көрсету процесін тиімді етеді.

1.2 Қайырымдылық жобаларын басқару ерекшеліктері

Қайырымдылық жобаларын басқару – бірнеше кезеңнен тұратын күрделі процесс. Бұл процесс көмекке мұқтаж адам немесе ұйым туралы ақпарат жинаудан басталады. Одан кейін мәліметтер тексеріліп, нақты қажеттілік анықталады. Кейін жоба ретінде жарияланып, оған қажетті қаражат немесе заттай көмек мөлшері белгіленеді. Жоба белсенді кезеңде демеушілерден үлес қабылданады, ал соңында көмек көрсетіліп, нәтиже жарияланады.

Қайырымдылық жобаларын басқарудағы басты ерекшелік – сенімділік пен ашықтықтың жоғары деңгейде болуы. Себебі көмек көрсетуші адам өз қаражатының немесе ұсынған көмегінің нақты мақсатқа жұмсалатынына сенуі керек. Сондықтан платформада әр жобаның сипаттамасы, категориясы, қаласы, маңыздылық деңгейі, мақсатты сомасы және жиналған сомасы нақты көрсетілуі тиіс.

Жобаларды басқаруда мәртебелердің болуы маңызды. Мысалы, жоба “дайындалуда”, “белсенді жинақ”, “уақытша тоқтатылған” немесе “аяқталған жоба” деген мәртебелерге ие болуы мүмкін. Бұл пайдаланушыларға жобаның қай кезеңде тұрғанын түсінуге көмектеседі. Белсенді жобаларға үлес қосуға болады, ал аяқталған жобалар нәтижені көрсету үшін сақталады.

Қайырымдылық жобалары әртүрлі бағыттарға бөлінеді. Мысалы, азық-түлік, денсаулық, білім, киім, тұрғын үй немесе басқа да көмек түрлері болуы

мүмкін. Категориялар жобаларды реттеуге және пайдаланушыға қажетті жобаны тез табуға мүмкіндік береді. Сонымен қатар қала немесе аймақ бойынша сүзгілеу де маңызды, себебі кейбір демеушілер өз өңіріндегі жобаларға көмектескісі келеді.

Жобаларды басқарудың тағы бір ерекшелігі – жоба прогресін көрсету. Егер жобаның мақсатты сомасы және жиналған сомасы көрсетілсе, жүйе пайыздық прогресті есептей алады. Бұл пайдаланушыға жобаның қаншалықты орындалғанын бірден түсінуге мүмкіндік береді. Прогресс көрсеткіші мотивациялық әсер береді, себебі адамдар мақсатқа қанша қалғанын көріп, үлес қосуға ынталануы мүмкін.

Қайырымдылық жобаларында жаңалықтар немесе есептер бөлімі де маңызды. Жоба бойынша қандай жұмыс атқарылды, қандай көмек берілді, қандай кезең аяқталды деген ақпараттар жаңалық ретінде жариялануы мүмкін. Бұл жүйенің ашықтығын арттырады және демеушілермен кері байланысты күшейтеді.

Жобаларды басқаруда модератор рөлі ерекше орын алады. Модератор өтініштерді қарайды, жобаларды жариялайды, деректерді түзетеді, мәртебелерді өзгертеді және жоба жаңалықтарын енгізеді. Сондықтан веб-платформада ыңғайлы әкімші панелі болуы қажет. Admin панель арқылы барлық негізгі деректерді басқару жобаның сапалы жұмыс істеуіне әсер етеді.

Осылайша қайырымдылық жобаларын басқару тек ақпарат жариялау емес, ол өтініш қабылдау, тексеру, жоба құру, көмек жинау, прогресті бақылау, нәтиже көрсету және есеп жүргізу кезеңдерін қамтиды. Веб-платформа осы кезеңдердің барлығын бір жүйеге біріктіруге мүмкіндік береді.

1.3 Веб-платформаға қойылатын негізгі талаптар

Қайырымдылық жобаларын басқаруға арналған веб-платформа пайдаланушыға ыңғайлы, қауіпсіз, түсінікті және функционалды болуы тиіс. Мұндай жүйе әлеуметтік бағытта қолданылатындықтан, оның сенімді жұмыс істеуі ерекше маңызды. Пайдаланушы платформаға кірген кезде қажетті ақпаратты тез тауып, жобаға үлес қосу немесе көмек сұрау әрекетін оңай орындауы керек.

Ең алдымен, веб-платформаға қойылатын функционалдық талаптарды қарастыру қажет. Жүйеде қайырымдылық жобаларын қосу, өңдеу, мәртебесін өзгерту және жою мүмкіндігі болуы керек. Әр жоба атауы, сипаттамасы, толық оқиғасы, категориясы, қаласы, маңыздылығы, мақсатты сомасы, жиналған сомасы, мерзімі және байланыс ақпараты сияқты деректерді қамтуы тиіс.

Платформада жобалар каталогы болуы маңызды. Каталог арқылы пайдаланушы барлық белсенді жобаларды көріп, іздеу және сүзгілеу мүмкіндігін қолдана алады. Сүзгілеу категория, қала, маңыздылық немесе мәртебе бойынша жүргізілуі мүмкін. Бұл пайдаланушының өзіне қажетті жобаны жылдам табуына көмектеседі.

Көмекке өтініш беру функционалы да негізгі талаптардың бірі болып табылады. Көмек сұраушы арнайы форма арқылы аты-жөнін, телефон нөмірін, email мекенжайын, қаласын, көмек түрін, басымдық деңгейін және жағдай сипаттамасын енгізеді. Бұл деректер деректер қорында сақталып, модератор тарапынан қаралады.

Үлес қосу функционалы платформадағы маңызды модульдердің бірі. Көмек көрсетуші белгілі бір жобаны таңдап, өзінің байланыс деректерін, көмек түрін, сомасын және қосымша хабарламасын жібере алады. Егер көмек түрі қаржылай болса, жүйе жиналған соманы автоматты түрде жаңарта алады. Бұл қайырымдылық жобасының прогресін нақты көрсетуге мүмкіндік береді.

Веб-платформада әкімші панелі болуы міндетті. Admin панель арқылы модератор жобаларды, өтініштерді, үлес қосу жазбаларын және жаңалықтарды басқарады. Сонымен қатар іздеу, сүзгілеу және деректерді топтау мүмкіндіктері модератордың жұмысын жеңілдетеді. Қазақ тілді admin панель жүйені жергілікті пайдаланушыларға түсінікті етеді.

Функционалдық емес талаптарға жүйенің жылдамдығы, қауіпсіздігі, адаптивтілігі және кеңейтілу мүмкіндігі жатады. Веб-бет әртүрлі құрылғыларда дұрыс ашылуы керек. Формалар дұрыс тексеріліп, қате деректердің сақталуына жол бермеуі қажет. Жүйе болашақта жаңа модульдер қосуға ыңғайлы архитектурада жасалуы тиіс.

Ақпараттық қауіпсіздік те маңызды талаптардың бірі. Пайдаланушы енгізген жеке деректер қорғалуы керек. Django фреймворкінде CSRF қорғанысы, авторизация, сессиялар және admin панельге кіру механизмдері қарастырылған. Бұл құралдар веб-платформаның қауіпсіздігін арттырады.

Сонымен қатар платформада статистикалық бөлім немесе dashboard болуы пайдалы. Dashboard арқылы белсенді жобалар саны, аяқталған жобалар, қабылданған өтініштер, үлес қосу жазбалары және жиналған қаражат туралы жалпы мәліметтер көрсетіледі. Мұндай бөлім модераторға платформаның ағымдағы жағдайын бақылауға мүмкіндік береді.

Қорытындылай келе, қайырымдылық веб-платформасына қойылатын негізгі талаптар – деректерді жүйелі сақтау, пайдаланушыға ыңғайлы интерфейс ұсыну, іздеу және сүзгілеу мүмкіндігін беру, көмек сұрау мен үлес қосу процестерін автоматтандыру, қауіпсіздік пен ашықтықты қамтамасыз ету. Осы талаптар орындалған жағдайда платформа қайырымдылық жобаларын басқарудың тиімді құралына айналады.

2 ВЕБ-ПЛАТФОРМАНЫ ЖОБАЛАУ

2.1 Жүйенің мақсаты, міндеттері және пайдаланушы рөлдері

Веб-платформаны жобалау кезеңі бағдарламалық өнімді әзірлеудегі ең маңызды сатылардың бірі болып табылады. Себебі кез келген ақпараттық жүйенің сапасы оның алдын ала қаншалықты дұрыс жоспарланғанына, құрылымы қаншалықты нақты анықталғанына және пайдаланушы қажеттіліктері қаншалықты ескерілгеніне байланысты. Қайырымдылық жобаларын басқаруға арналған веб-платформа әлеуметтік бағыттағы жүйе болғандықтан, оны жобалау барысында тек техникалық талаптарды ғана емес, сонымен қатар пайдаланушылардың сенімі, ақпараттың ашықтығы, деректердің қауіпсіздігі және қолданудың қарапайымдылығы сияқты факторларды да ескеру қажет.

Қайырымдылық платформасының негізгі міндеті – көмекке мұқтаж адамдар, көмек көрсетушілер, еріктілер және модераторлар арасындағы байланысты жеңілдету. Мұндай жүйеде әрбір пайдаланушының әрекеті белгілі бір логикаға бағынуы тиіс. Мысалы, көмек сұраушы өтініш қалдырады, модератор өтінішті қарап шығады, қажет болған жағдайда оны қайырымдылық жобасына айналдырады, ал демеуші немесе ерікті белсенді жобалар арасынан өзіне жақын бағытты таңдап, үлес қосу туралы ұсыныс жібереді.

Жүйені жобалау барысында веб-платформаның негізгі беттері, деректер қорының құрылымы, пайдаланушы рөлдері, форма өрістері, жоба мәртебелері, іздеу және сүзгілеу логикасы, әкімші панелінің мүмкіндіктері және ақпараттық қауіпсіздік талаптары анықталды. Платформа көп бетті құрылымға ие: басты бет, жобалар каталогы, жоба туралы толық ақпарат беті, көмекке өтініш беру беті, үлес қосу беті, dashboard бөлімі және платформа туралы бет қарастырылды.

Бұл бөлімде жүйенің мақсаты мен міндеттері, пайдаланушы рөлдері, деректер қорының құрылымы, жүйе архитектурасы және интерфейс құрылымы толық сипатталады.

Қайырымдылық жобаларын басқаруға арналған веб-платформаның негізгі мақсаты – қайырымдылық қызметін цифрлық ортада ұйымдастыру, көмек сұраушылар мен көмек көрсетушілер арасындағы байланысты жеңілдету және қайырымдылық жобалары туралы ақпаратты ашық түрде ұсыну. Бұл жүйе арқылы қайырымдылық жобаларын жариялау, жиналған қаражатты бақылау, көмекке өтініш қабылдау, үлес қосу ұсыныстарын тіркеу және жоба нәтижесін көрсету мүмкін болады.

Платформаның мақсаты тек ақпарат жариялаумен шектелмейді. Жүйе қайырымдылық процесінің бірнеше кезеңін қамтиды. Бірінші кезеңде көмекке мұқтаж адам немесе ұйым арнайы форма арқылы өтініш қалдырады. Екінші кезеңде модератор бұл өтінішті қарап, оның шынайылығын тексереді. Үшінші кезеңде өтініш негізінде қайырымдылық жобасы жарияланады. Төртінші

кезеңде демеушілер немесе еріктілер жобаға үлес қосады. Соңғы кезеңде жоба аяқталып, оның нәтижесі көрсетіледі.

Жүйенің негізгі міндеттері төмендегідей:

- қайырымдылық жобалары туралы ақпаратты бір ортада жинақтау;
- көмекке өтініш беру процесін жеңілдету;
- қайырымдылық жобаларын категория, қала, мәртебе және маңыздылық бойынша бөлу;
- әр жоба бойынша мақсатты сома мен жиналған соманы көрсету;
- жоба прогресін пайыздық түрде есептеу;
- үлес қосу өтініштерін қабылдау және сақтау;
- жоба жаңалықтарын жариялау;
- модераторға арналған әкімші панелін ұйымдастыру;
- платформа бойынша статистикалық мәліметтерді көрсету;
- пайдаланушы деректерін қауіпсіз сақтау;
- жүйені болашақта кеңейтуге қолайлы етіп жобалау.

Жүйеде бірнеше пайдаланушы рөлі қарастырылған. Әр рөлдің өз міндеті мен мүмкіндігі бар. Пайдаланушы рөлдерін нақты бөлу жүйенің түсінікті жұмыс істеуіне және ақпараттың дұрыс өңделуіне мүмкіндік береді.

Бірінші рөл – көмек сұраушы. Бұл рөлге қиын жағдайға тап болған азамат немесе көмекке мұқтаж отбасы жатады. Көмек сұраушы платформадағы арнайы форма арқылы аты-жөнін, телефон нөмірін, email мекенжайын, қаласын, көмек түрін, басымдық деңгейін және жағдай сипаттамасын енгізеді. Бұл мәліметтер жүйеде көмек өтініші ретінде сақталады. Көмек сұраушының басты әрекеті – өтініш қалдыру.

Екінші рөл – көмек көрсетуші. Бұл рөлге демеушілер, еріктілер немесе қайырымдылық жасауға ниетті кез келген пайдаланушы жатады. Көмек көрсетуші белсенді жобаларды қарап, категория немесе қала бойынша өзіне жақын жобаны таңдай алады. Жоба detail бетінде ол жоба сипаттамасын, мақсатты соманы, жиналған соманы, қалған қаражатты және прогресс көрсеткішін көреді. Егер көмектескісі келсе, үлес қосу формасын толтырады.

Үшінші рөл – ерікті. Ерікті көмек көрсетушінің бір түрі ретінде қарастырылуы мүмкін. Бірақ оның ерекшелігі – ол міндетті түрде қаржылай көмек көрсетпейді. Ерікті заттай көмек ұсынуы, кеңес беруі немесе нақты ұйымдастыру жұмыстарына қатысуы мүмкін. Сондықтан жүйеде үлес қосу формасында көмек түрлері бірнеше нұсқаға бөлінеді: қаржылай көмек, заттай көмек, ерікті болу және кеңес беру.

Төртінші рөл – модератор немесе әкімші. Бұл рөл жүйедегі ақпаратты басқаруға жауап береді. Модератор admin панель арқылы қайырымдылық жобаларын қосады, көмек өтініштерін қарайды, үлес қосу жазбаларын тексереді, жоба жаңалықтарын енгізеді және жобаның мәртебесін өзгертеді. Модератордың жұмысы платформаның сенімділігіне тікелей әсер етеді, себебі ол жарияланатын ақпараттың дұрыстығын бақылауы қажет.

Платформадағы пайдаланушы әрекеттерін төмендегідей сипаттауға болады. Көмек сұраушы “Көмек сұрау” бетіне кіріп, өтініш формасын толтырады. Бұл өтініш деректер қорына сақталады. Модератор admin панель арқылы өтінішті қарап шығады. Егер өтініш расталса, ол жаңа қайырымдылық жобасын құрады. Жоба “белсенді” мәртебесін алғаннан кейін жалпы каталогта көрсетіледі. Көмек көрсетуші каталогтан жобаны таңдап, үлес қосу формасын толтырады. Егер үлес қаржылай болса, жүйе жиналған соманы жаңартады. Жоба аяқталған кезде модератор оның мәртебесін “аяқталған” деп өзгертеді.

Жүйенің мақсаты мен рөлдерін нақты анықтау веб-платформаның әрі қарайғы құрылымын дұрыс жобалауға негіз болады. Егер пайдаланушы рөлдері анық болмаса, жүйедегі әрекеттер де түсініксіз болады. Сондықтан бұл дипломдық жұмыста әрбір рөлдің міндеті нақты бөлініп, соған сәйкес беттер мен деректер қоры құрылымы жасалды.

2.2 Деректер қорының құрылымын жобалау

Деректер қоры – кез келген ақпараттық жүйенің негізгі бөлігі. Қайырымдылық жобаларын басқаруға арналған веб-платформада барлық маңызды ақпарат деректер қорында сақталады. Оған қайырымдылық жобалары, көмек өтініштері, үлес қосу жазбалары, жоба жаңалықтары және жүйелік уақыт белгілері жатады. Деректер қорын дұрыс жобалау жүйенің тұрақты, түсінікті және кеңейтуге қолайлы болуына әсер етеді.

Бұл жобада деректер қоры SQLite негізінде ұйымдастырылды. SQLite шағын және орта көлемдегі жобаларға ыңғайлы, себебі бөлек сервер орнатуды қажет етпейді және деректер бір файлда сақталады. Оқу жобасы үшін бұл өте қолайлы шешім болып табылады. Сонымен қатар Django ORM арқылы модельдер құру деректер қорымен жұмыс істеуді жеңілдетеді [1].

Жүйеде негізгі төрт модель қарастырылды:

- CharityProject;
- HelpRequest;
- Contribution;
- ProjectUpdate.

CharityProject моделі қайырымдылық жобалары туралы ақпаратты сақтайды. Бұл модель платформаның негізгі объектісі болып табылады. Әр жоба атауынан, қысқаша сипаттамасынан, толық оқиғасынан, категориясынан, қаласынан, маңыздылық деңгейінен, мақсатты сомасынан, жиналған сомасынан, мерзімінен, байланыс телефонынан, жауапты адамнан, мәртебесінен және қаралым санынан тұрады.

CharityProject моделіндегі негізгі өрістердің бірі – title. Бұл өріс жобаның атауын сақтайды. Мысалы, “Алматыдағы көпбалалы отбасыға қысқы киім” немесе “Шымкентте ем қабылдап жатқан балаға дәрі-дәрмек” сияқты атаулар болуы мүмкін. description өрісі жобаның қысқаша сипаттамасын сақтайды, ал story өрісі толық оқиғаны сипаттауға арналған.

category өрісі жобаны белгілі бір бағытқа бөлуге мүмкіндік береді. Жүйеде азық-түлік, денсаулық, білім, киім, тұрғын үй және басқа категориялар қарастырылған. Бұл пайдаланушыға қажетті жобаны жылдам табуға көмектеседі. Мысалы, бір адам тек денсаулық саласындағы жобаларға көмектескісі келсе, ол категория сүзгісін қолдана алады.

city өрісі жобаның қай қалада немесе аймақта орналасқанын көрсетеді. Бұл да маңызды, себебі кейбір демеушілер өз қаласындағы немесе өз өңіріндегі жобаларға көмектесуді қалайды. urgency өрісі жобаның маңыздылық деңгейін білдіреді. Мысалы, төмен, орташа, жоғары немесе шұғыл деңгейлер болуы мүмкін.

target_amount және collected_amount өрістері жобаның қаржылық көрсеткіштерін сақтайды. target_amount – жобаға қажет жалпы сома, ал collected_amount – қазіргі уақытқа дейін жиналған сома. Осы екі өріс негізінде жүйе жоба прогресін есептейді. Егер мақсатты сома 500 000 теңге, ал жиналған сома 250 000 теңге болса, прогресс 50 пайызды құрайды.

status өрісі жобаның ағымдағы мәртебесін көрсетеді. Жоба дайындалуда, белсенді, уақытша тоқтатылған немесе аяқталған болуы мүмкін. Бұл өріс платформада қандай жобаларды көрсету керектігін анықтауға көмектеседі. Мысалы, басты бетте көбіне белсенді жобалар көрсетіледі.

HelpRequest моделі көмекке өтініш қалдырған адамдар туралы ақпаратты сақтайды. Бұл модельде аты-жөні, телефон нөмірі, email, қала, көмек түрі, басымдық деңгейі, хабарлама мәтіні, өтініш мәртебесі және модератор жазбасы сақталады. Көмек сұраушы форма толтырған кезде осы модельге жаңа жазба қосылады.

HelpRequest моделіндегі request_type өрісі көмек түрін көрсетеді. Мысалы, азық-түлік, денсаулық, білім, киім, тұрғын үй немесе басқа көмек түрі болуы мүмкін. priority өрісі өтініштің маңыздылығын білдіреді. Егер жағдай өте шұғыл болса, өтініш “шұғыл” ретінде белгіленеді. Бұл модераторға қай өтінішті бірінші қарау керектігін анықтауға көмектеседі.

status өрісі өтініштің өңделу кезеңін көрсетеді. Жаңа, тексерілуде, қабылданды, қабылданбады немесе жабылды деген мәртебелер болуы мүмкін. Мұндай құрылым өтініштерді жүйелі түрде басқаруға мүмкіндік береді.

Contribution моделі көмек көрсетушілердің үлес қосу өтініштерін сақтайды. Бұл модель нақты бір CharityProject жазбасымен байланысады. Яғни әр үлес белгілі бір жобаға тиесілі болады. Contribution моделінде жоба, көмек көрсетушінің аты-жөні, телефоны, email мекенжайы, көмек түрі, сома, хабарлама, мәртебе және анонимділік белгісі сақталады.

Contribution моделіндегі method өрісі көмек түрін анықтайды. Қаржылай көмек, заттай көмек, ерікті болу немесе кеңес беру нұсқалары қарастырылған. Егер көмек түрі қаржылай болса және енгізілген сома нөлден жоғары болса, жүйе бұл соманы жобаның жиналған сомасына қосады. Осылайша жоба прогресі автоматты түрде жаңарады.

is_anonymous өрісі көмек көрсетушінің аты-жөнін жарияламау мүмкіндігін береді. Бұл пайдаланушы құпиялылығын сақтауға көмектеседі.

applied_to_total өрісі үлес сомасының жоба сомасына қосылған-қосылмағанын көрсетеді. Бұл қайталама қосу қателігін болдырмауға арналған.

ProjectUpdate моделі жоба жаңалықтарын сақтау үшін қолданылады. Әр жаңалық белгілі бір жобаға тиесілі болады. Бұл модельде жоба, жаңалық тақырыбы, жаңалық мәтіні және сайтта көрсету белгісі сақталады. Жоба жаңалықтары арқылы модератор атқарылған жұмыстарды, жеткізілген көмекті немесе жаңа ақпаратты жариялай алады.

Деректер қорындағы байланыстар да маңызды. CharityProject пен Contribution арасында “бір көпке” байланысы бар: бір жобаға бірнеше үлес қосу жазбасы тіркелуі мүмкін. CharityProject пен ProjectUpdate арасында да “бір көпке” байланысы бар: бір жобаға бірнеше жаңалық қосуға болады. HelpRequest жеке модель ретінде сақталады, себебі әр өтініш кейін жобаға айналуы немесе тек модератор қарауында қалуы мүмкін.

Деректер қорын жобалау кезінде индекстер де қарастырылды. Мысалы, жобаларда status және category өрістері бойынша индекс қолдану іздеу мен сүзгілеуді жылдамдатады. HelpRequest моделінде status және priority өрістері бойынша индекс өтініштерді тез табуға көмектеседі. Contribution моделінде project және created_at өрістері бойынша индекс жобаға қатысты үлестерді жылдам шығаруға мүмкіндік береді.

Деректер қорының осындай құрылымы қайырымдылық веб-платформасының негізгі процестерін толық қамтиды. Жоба туралы ақпарат, көмек өтініштері, үлес қосу жазбалары және жаңалықтар бір-бірімен логикалық байланыста сақталады. Бұл жүйені әрі қарай кеңейтуге де ыңғайлы. Мысалы, болашақта пайдаланушы кабинеті, төлем тарихы, файл жүктеу немесе есеп беру модулі қосылса, осы модельдер негізінде жаңа кестелер қосуға болады.

2.3 Жүйе архитектурасы және интерфейс құрылымы

Жүйе архитектурасы – веб-платформаның ішкі құрылымын, модульдер арасындағы байланысты және ақпараттың қалай өңделетінін сипаттайды. Қайырымдылық жобаларын басқаруға арналған веб-платформа Django фреймворкі негізінде жасалғандықтан, оның архитектурасы Model–Template–View қағидасына сүйенеді.

Django жүйесінде Model деректер қорымен жұмыс істеуге жауап береді. Бұл қабатта CharityProject, HelpRequest, Contribution және ProjectUpdate сияқты модельдер орналасады. Әр модель деректер қорындағы белгілі бір кестеге сәйкес келеді. Модельдер арқылы деректерді қосу, оқу, өзгерту және жою әрекеттері орындалады [1, 2].

View қабаты пайдаланушы сұраныстарын өңдейді. Мысалы, пайдаланушы /projects/ бетіне кірсе, project_list_view функциясы іске қосылады. Бұл функция деректер қорынан жобаларды алып, қажет болса іздеу және сүзгілеу шарттарын қолданады. Кейін алынған деректер template файлына жіберіледі.

Template кабаты пайдаланушыға көрсетілетін HTML беттерді қалыптастырады. Бұл жобада бірнеше template қолданылды: басты бет, жобалар тізімі, жоба detail беті, көмек сұрау беті, үлес қосу беті, dashboard және about беті. Барлық беттер ортақ base.html шаблонна негізделген. Бұл навигация мен дизайнның біркелкі болуын қамтамасыз етеді.

Жүйенің ақпарат ағынын былай сипаттауға болады. Пайдаланушы браузерден белгілі бір URL мекенжайына сұраныс жібереді. Django URL маршруты бұл сұранысты тиісті view функциясына бағыттайды. View функциясы қажет деректерді модельдер арқылы деректер қорынан алады немесе форма арқылы келген мәліметтерді сақтайды. Содан кейін view алынған деректерді template-ке context ретінде береді. Template HTML бет құрастырып, браузерге қайтарады.

Платформадағы негізгі URL бағыттары бірнеше бөлімнен тұрады. Басты бет / мекенжайында орналасқан. Жобалар каталогы /projects/ бетінде көрсетіледі. Нақты жоба туралы толық ақпарат /projects/<id>/ мекенжайында ашылады. Көмек сұрау беті /request-help/, үлес қосу беті /contribute/, аналитикалық бөлім /dashboard/, ал платформа туралы ақпарат /about/ мекенжайында орналасқан.

Басты бет интерфейсі пайдаланушыға платформаның мақсатын бірден түсіндіруге бағытталған. Онда платформа атауы, қысқаша сипаттама, негізгі әрекеттерге арналған батырмалар, ұсынылған жобалар және жалпы статистика көрсетіледі. Бұл бет пайдаланушыны жүйемен таныстыру қызметін атқарады.

Жобалар каталогы бетінде іздеу және сүзгілеу элементтері орналасқан. Пайдаланушы жоба атауы немесе сипаттамасы бойынша іздей алады, категорияны таңдай алады, қала енгізе алады, маңыздылық деңгейін және мәртебені белгілей алады. Мұндай интерфейс көп жоба болған жағдайда қажетті ақпаратты тез табуға көмектеседі.

Жоба detail беті ең маңызды беттердің бірі болып табылады. Бұл бетте жобаның толық атауы, сипаттамасы, толық оқиғасы, категориясы, мәртебесі, маңыздылық деңгейі, қала, қаралым саны, мерзімі, мақсатты сома, жиналған сома және прогресс көрсеткіші беріледі. Сонымен қатар осы бетте үлес қосу формасы орналасады. Яғни пайдаланушы жоба туралы ақпаратты оқып, сол жерде көмек ұсына алады.

Көмек сұрау беті қарапайым әрі түсінікті формаға негізделген. Бұл бетте пайдаланушы өзінің байланыс деректерін және қандай көмек қажет екенін енгізеді. Форманың қарапайым болуы маңызды, себебі көмек сұраушы адам күрделі интерфейстен қиналмауы керек.

Үлес қосу беті көмек көрсетушілерге арналған. Бұл бетте пайдаланушы белсенді жобалардың бірін таңдап, көмек түрін, сомасын және байланыс деректерін енгізеді. Егер пайдаланушы аты-жөнін көрсеткісі келмесе, анонимділік белгісін таңдай алады.

Dashboard беті модераторға немесе жоба жетекшісіне арналған ақпараттық бөлім ретінде жасалған. Онда белсенді жобалар саны, аяқталған жобалар саны, көмек өтініштері, үлес қосу жазбалары және жиналған қаражат

туралы мәліметтер көрсетіледі. Сонымен қатар соңғы өтініштер мен соңғы үлестер кесте түрінде беріледі [3].

Интерфейс құрылымында Bootstrap кітапханасы қолданылды. Бұл әртүрлі экран өлшемдеріне бейімделетін дизайн жасауға мүмкіндік береді. Мысалы, компьютерде бірнеше баған болып көрінетін карточкалар мобильді құрылғыда бір бағанға орналасады. Бұл пайдаланушыға телефоннан да, компьютерден де ыңғайлы жұмыс істеуге жағдай жасайды [4].

Жүйе интерфейсінде карточкалар, батырмалар, прогресс жолақтары, кестелер және форма элементтері қолданылады. Қайырымдылық жобалары карточка түрінде көрсетіледі, себебі карточкада атау, қысқаша сипаттама, категория, қала, жиналған сома және прогресс сияқты негізгі ақпаратты ықшам көрсетуге болады.

Пайдаланушы тәжірибесін жақсарту үшін навигация барлық бетте бірдей орналасады. Жоғарғы мәзірде “Жобалар”, “Көмек сұрау”, “Үлес қосу”, “Dashboard”, “Платформа” және “Admin” сілтемелері бар. Бұл пайдаланушыға кез келген беттен қажетті бөлімге тез өтуге мүмкіндік береді.

Жүйе архитектурасы мен интерфейс құрылымы бір-бірімен тығыз байланысты. Деректер қорында сақталған ақпарат view арқылы өңделіп, template арқылы пайдаланушыға түсінікті түрде көрсетіледі. Осылайша веб-платформа қайырымдылық жобаларын басқаруға арналған толыққанды ақпараттық жүйе ретінде жұмыс істейді.

Жобаланған архитектура болашақта жүйені кеңейтуге мүмкіндік береді. Мысалы, онлайн төлем жүйесін қосу үшін Contribution моделін төлем статустарымен толықтыруға болады. Пайдаланушы кабинетін қосу үшін Django authentication жүйесін кеңейтуге болады. Файл жүктеу мүмкіндігін қосу үшін жобаға ImageField немесе FileField өрістерін енгізуге болады. Бұл платформаның құрылымы икемді және дамытуға қолайлы екенін көрсетеді.

3 ВЕБ-ПЛАТФОРМАНЫ ІСКЕ АСЫРУ

3.1 Django жобасының құрылымы және негізгі модульдері

Веб-платформаны іске асыру кезеңі жобалау бөлімінде анықталған талаптарды нақты бағдарламалық өнімге айналдыруға бағытталды. Бұл кезеңде жүйенің серверлік бөлігі, деректер қоры, пайдаланушы интерфейсі, формалар, навигация, әкімші панелі және тестілеу құралдары әзірленді. Қайырымдылық жобаларын басқаруға арналған веб-платформа Django фреймворкі негізінде жасалды. Django фреймворкін таңдау себебі – оның деректер қорымен жұмыс істеуге арналған дайын ORM жүйесі, қауіпсіздік механизмдері, шаблондармен жұмыс істеу мүмкіндігі және дайын әкімші панелі бар.

Жобаның негізгі мақсаты – қайырымдылық жобаларын жариялау, көмекке өтініш қабылдау, үлес қосу ұсыныстарын тіркеу және жобалардың орындалу барысын бақылау мүмкіндігін беретін ыңғайлы веб-жүйе жасау. Осы мақсатқа сәйкес платформа бірнеше негізгі бөлімнен тұрады: басты бет, жобалар каталогы, жоба туралы толық ақпарат беті, көмек сұрау беті, үлес қосу беті, dashboard бөлімі, платформа туралы бет және әкімші панелі. Әр бөлімнің өзіндік міндеті бар және олар бір-бірімен логикалық байланыста жұмыс істейді.

Платформаны іске асыру барысында пайдаланушыға түсінікті интерфейс жасауға ерекше көңіл бөлінді. Себебі қайырымдылық платформасын әртүрлі деңгейдегі қолданушылар пайдалануы мүмкін: көмекке мұқтаж азаматтар, еріктілер, демеушілер, модераторлар немесе ұйым өкілдері. Сондықтан интерфейс күрделі емес, ақпаратты тез қабылдауға ыңғайлы және қазақ тілінде жасалды.

Django жобасының құрылымы бағдарламалық өнімді модульдерге бөліп ұйымдастыруға мүмкіндік береді. Бұл жобада негізгі конфигурациялық файлдар `charity_project` каталогында, ал платформаның негізгі логикасы `platforma` қосымшасында орналастырылды. Мұндай құрылым Django жобаларына тән және жүйені әрі қарай дамытуға ыңғайлы.

Жобаның жалпы құрылымы төмендегідей ұйымдастырылды:

- `manage.py` – Django жобасын басқаруға арналған негізгі файл;
- `charity_project/settings.py` – жобаның негізгі баптаулары орналасқан файл;
- `charity_project/urls.py` – URL маршруттары көрсетілген файл;
- `platforma/models.py` – деректер қоры модельдері орналасқан файл;
- `platforma/forms.py` – пайдаланушы формалары сипатталған файл;
- `platforma/views.py` – беттердің логикасы жазылған файл;
- `platforma/admin.py` – әкімші панелінің баптаулары орналасқан файл;
- `platforma/templates/platforma/` – HTML шаблондар сақталған каталог;
- `platforma/management/commands/seed_demo.py` – демо деректерді енгізуге арналған команда.

Жобаның негізгі қосымшасы ретінде platforma қолданылды. Бұл қосымшада қайырымдылық жобалары, көмек өтініштері, үлес қосу жазбалары және жоба жаңалықтары бойынша барлық негізгі логика жинақталған. Мұндай тәсіл жобаның құрылымын түсінікті етеді, себебі әр файл нақты бір қызмет атқарады [5, 6, 7, 8].

models.py файлында деректер қорының негізгі модельдері сипатталды. Жүйеде төрт негізгі модель қолданылды: CharityProject, HelpRequest, Contribution және ProjectUpdate. Бұл модельдер қайырымдылық платформасының негізгі ақпараттық объектілерін сипаттайды. Мысалы, CharityProject моделі қайырымдылық жобасы туралы деректерді сақтаса, HelpRequest моделі көмекке өтініш қалдырған пайдаланушы туралы ақпаратты сақтайды [9, 10].

forms.py файлында екі негізгі форма жасалды: көмекке өтініш беру формасы және үлес қосу формасы. Бұл формалар пайдаланушыдан дерек қабылдау үшін қолданылады. Формалар Django ModelForm негізінде құрылғандықтан, енгізілген деректер бірден тиісті модель құрылымымен байланысады. Бұл тәсіл кодты қысқартады және деректерді тексеруді жеңілдетеді.

views.py файлында веб-платформаның негізгі беттері үшін view функциялары жазылды. Әр view белгілі бір URL сұранысын өңдейді, деректер қорынан қажетті ақпаратты алады және оны HTML шаблонға жібереді. Мысалы, басты бетке арналған home_view, жобалар тізіміне арналған project_list_view, жоба detail бетіне арналған project_detail_view, көмек сұрау бетіне арналған request_help_view, үлес қосу бетіне арналған contribute_view және статистикаға арналған dashboard_view функциялары жасалды.

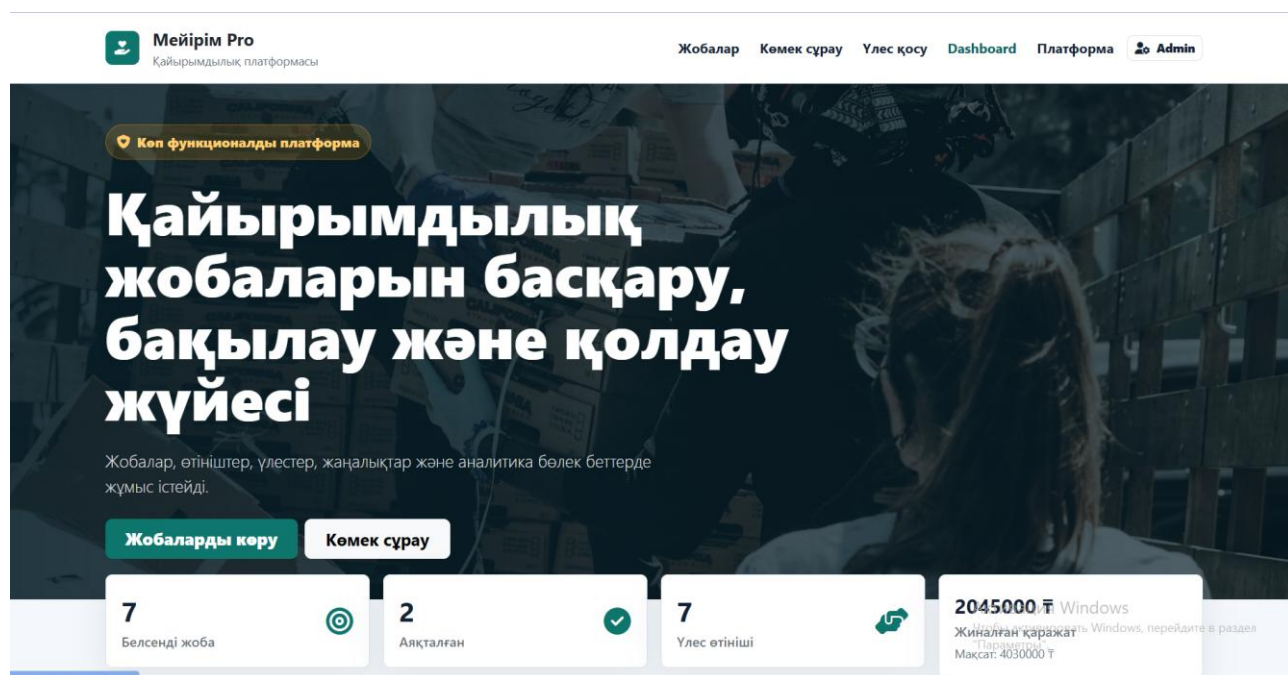
urls.py файлында барлық беттердің маршруттары анықталды. Жобада келесі негізгі URL бағыттары қолданылды:

- / – басты бет;
- /projects/ – жобалар каталогы;
- /projects/<id>/ – нақты жоба туралы толық ақпарат;
- /request-help/ – көмекке өтініш беру беті;
- /contribute/ – үлес қосу беті;
- /dashboard/ – аналитикалық көрсеткіштер беті;
- /about/ – платформа туралы бет;
- /admin/ – әкімші панелі.

Жобаның серверлік құрылымы Django қағидасына сәйкес Model–Template–View үлгісі бойынша ұйымдастырылды. Model деректермен жұмыс істейді, View бизнес логиканы орындайды, ал Template пайдаланушыға көрсетілетін HTML бетті қалыптастырады. Бұл архитектура жобаның әр бөлігін бөлек дамытуға мүмкіндік береді.

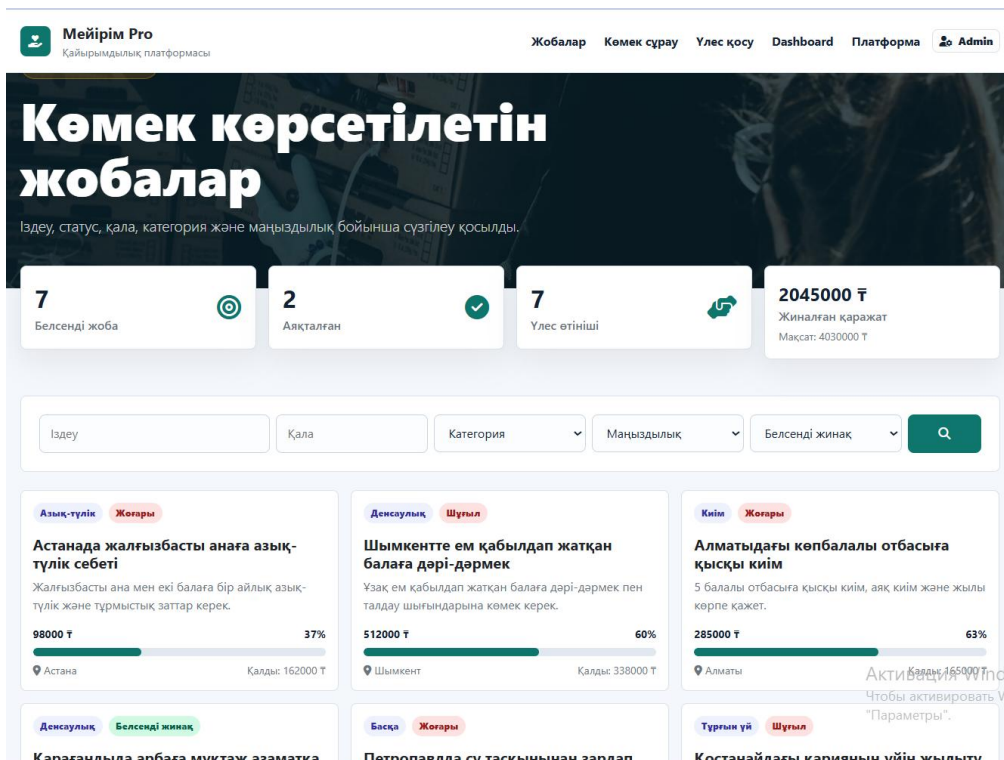
Жобаның басты беті платформаның жалпы концепциясын таныстыруға арналған. Мұнда пайдаланушы қайырымдылық платформасының мақсатын, белсенді жобаларды, жалпы статистиканы және негізгі әрекеттерді көре алады.

Басты бет арқылы пайдаланушы “Жобаларды көру”, “Көмек сұрау” немесе “Үлес қосу” бөлімдеріне өте алады. Басты бет экраны 1-суретте көрсетілген.



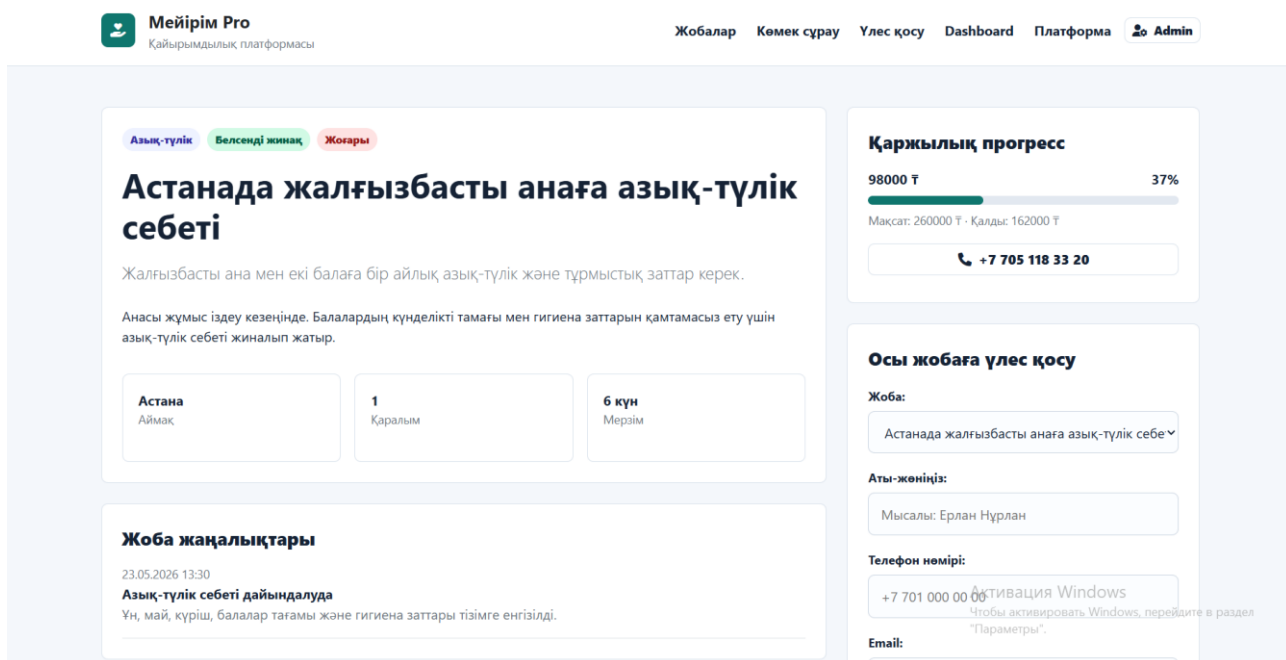
1-сурет. Веб-платформаның басты беті

Жобалар каталогы платформаның негізгі бөлімдерінің бірі болып табылады. Бұл бетте барлық белсенді немесе аяқталған қайырымдылық жобалары көрсетіледі. Пайдаланушы жобаларды категория, қала, мәртебе және маңыздылық деңгейі бойынша сүзгілей алады. Сонымен қатар іздеу өрісі арқылы жоба атауы немесе сипаттамасы бойынша қажетті ақпаратты табуға болады. Жобалар каталогы экраны 2-суретте көрсетілген.



2-сурет. Жобалар каталогы беті

Жоба туралы толық ақпарат бетінде нақты қайырымдылық жобасының барлық деректері беріледі. Мұнда жобаның атауы, сипаттамасы, толық оқиғасы, категориясы, қаласы, маңыздылық деңгейі, мақсатты сома, жиналған сома, қалған қаражат және прогресс көрсеткіші көрсетіледі. Сонымен қатар осы бетте пайдаланушы жобаға үлес қосу формасын толтыра алады. Жоба detail беті экраны 3-суретте көрсетілген.



3-сурет. Жоба туралы толық ақпарат беті

Осылайша Django жобасының құрылымы веб-платформаның барлық негізгі модульдерін жүйелі түрде ұйымдастыруға мүмкіндік берді. Әр модульдің жеке міндеті бар және олар бір-бірімен байланыса отырып толыққанды қайырымдылық платформасын құрайды.

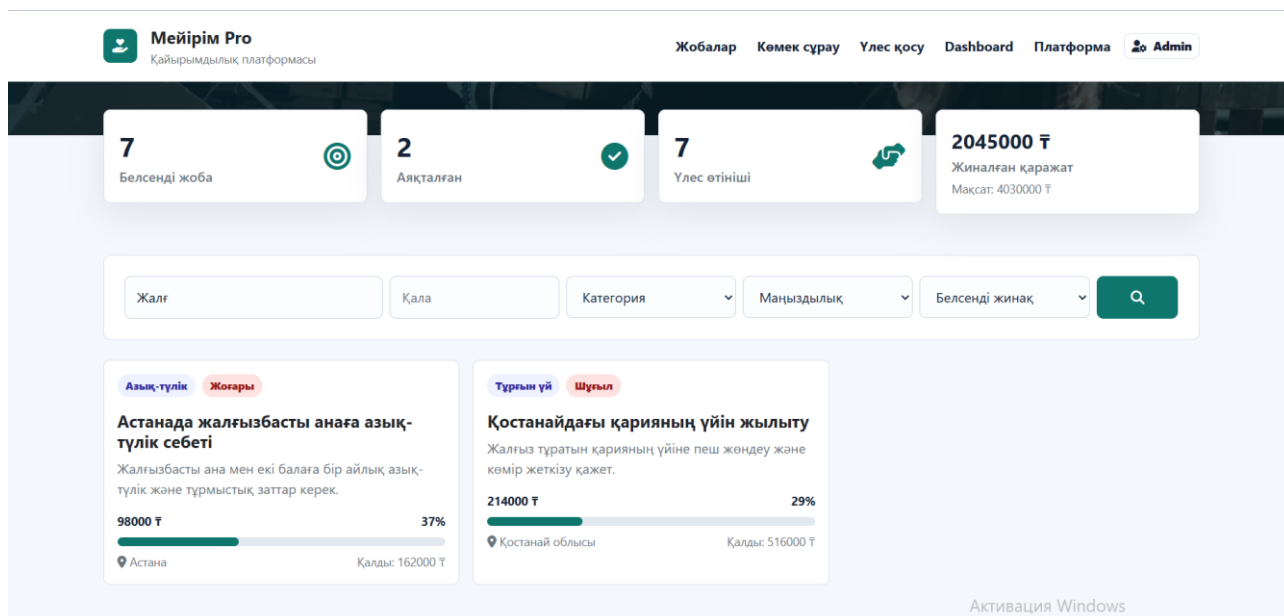
3.2 Негізгі функционалдарды әзірлеу

Веб-платформаның негізгі функционалдары пайдаланушының қайырымдылық жобаларымен жұмыс істеуін жеңілдетуге бағытталды. Жүйеде қарапайым ақпараттық беттер ғана емес, нақты әрекет жасауға мүмкіндік беретін бірнеше маңызды модуль іске асырылды. Оларға қайырымдылық жобаларын көру, іздеу және сүзгілеу, жобаға үлес қосу, көмекке өтініш беру, жоба жаңалықтарын көрсету, статистикалық мәліметтерді шығару және әкімші панелі арқылы деректерді басқару жатады.

Ең алдымен, қайырымдылық жобалары каталогы әзірленді. Бұл бөлімде жүйеде сақталған жобалар карточка түрінде көрсетіледі. Әр карточкада жобаның атауы, қысқаша сипаттамасы, категориясы, қаласы, жиналған сомасы және прогресс пайызы беріледі. Карточкалық құрылым пайдаланушыға бірнеше жобаны бір экранда салыстырып көруге мүмкіндік береді.

Жобалар каталогында іздеу және сүзгілеу функционалы қосылды. Пайдаланушы іздеу өрісіне кілт сөз енгізу арқылы жоба атауы немесе сипаттамасы бойынша іздей алады. Сонымен қатар категория, қала, маңыздылық және мәртебе бойынша сүзгілеу мүмкіндігі бар. Бұл функционал әсіресе жобалар саны көбейген кезде өте пайдалы. Мысалы, пайдаланушы тек өз қаласындағы немесе тек денсаулық саласындағы жобаларды көргісі келсе, сүзгілеу құралдарын қолданады.

Жобалар каталогының мазмұны қосымшаның жалпы концепциясымен үйлеседі. Мұнда пайдаланушы тек жобалардың тізімін ғана көріп қоймай, платформаның қандай әлеуметтік бағыттарға қызмет ететінін де түсінеді. Мысалы, азық-түлік, денсаулық, білім, киім және тұрғын үй сияқты категориялар арқылы жүйенің қайырымдылық саласындағы кең қолдану мүмкіндігі байқалады. Бұл шешім жаңа пайдаланушы үшін пайдалы, себебі ол платформадағы жобалар құрылымын бірден түсінеді. Жобалар каталогы және сүзгілеу бөлімі 4-суретте көрсетілген.



4-сурет. Жобаларды іздеу және сүзгілеу бөлімі

Келесі маңызды функционал – көмекке өтініш беру модулі. Бұл модуль көмекке мұқтаж азаматтарға арналған. Пайдаланушы арнайы форма арқылы өзінің аты-жөнін, телефон нөмірін, email мекенжайын, қаласын, көмек түрін, басымдық деңгейін және жағдай сипаттамасын енгізеді. Форма жіберілгеннен кейін деректер HelpRequest моделіне сақталады [11].

Көмекке өтініш беру беті қарапайым және түсінікті түрде жасалды. Себебі бұл бетті қиын жағдайға тап болған адамдар қолдануы мүмкін, сондықтан форма күрделі болмауы тиіс. Өрістердің атаулары қазақ тілінде берілген және пайдаланушы қандай ақпарат енгізу керектігін оңай түсінеді. Өтініш сәтті жіберілгеннен кейін жүйе пайдаланушыға хабарлама көрсетеді. Көмекке өтініш беру беті 5-суретте көрсетілген.

Көмекке өтініш қалдыру

Аты-жөніңіз:

Телефон нөмірі:

Email:

Қала/аймақ:

Көмек түрі:

Маңыздылығы:

Жағдайды қысқаша сипаттаңыз:

Соңғы өтініштер

АТЫ	ТҮРІ	СТАТУС	КҮНІ
Ермек Асан	Киім	Жаңа	23.05
Салтанат Ораз	Тұрғын үй	Тексерілуде	23.05
Аружан Нұрбек	Білім	Қабылданды	23.05
Марат Қали	Денсаулық	Жаңа	23.05
Гүлнәр Сарсен	Азық-түлік	Тексерілуде	23.05
aaa	Басқа	Жаңа	22.05

5-сурет. Көмекке өтініш беру формасы

Үлес қосу модулі көмек көрсетушілерге арналған негізгі бөлімдердің бірі болып табылады. Бұл бөлім арқылы пайдаланушы платформада жарияланған белсенді қайырымдылық жобаларының бірін таңдап, сол жобаға қандай түрде көмектесе алатынын көрсете алады. Үлес қосу формасында пайдаланушы өзінің аты-жөнін, телефон нөмірін, email мекенжайын, көмек түрін, сомасын және қосымша хабарламасын енгізеді. Мұндай құрылым көмек көрсетуші туралы бастапқы ақпаратты жүйелі түрде жинауға және модератордың кейін байланысқа шығуына мүмкіндік береді.

Көмек түрі бірнеше нұсқадан тұрады: қаржылай көмек, заттай көмек, ерікті болу және кеңес беру. Бұл платформаның тек ақша жинауға ғана бағытталмағанын көрсетеді. Қайырымдылық әрқашан қаржылай қолдаумен шектелмейді, кей жағдайда адамдар киім, азық-түлік, дәрі-дәрмек, оқу құралдары сияқты заттай көмек ұсына алады. Сонымен қатар кейбір пайдаланушылар ерікті ретінде жеткізу, ұйымдастыру немесе ақпарат тарату жұмыстарына қатыса алады. Ал кеңес беру нұсқасы белгілі бір сала мамандарының тәжірибесін қажет ететін жағдайларда пайдалы болуы мүмкін.

Үлес қосу модулі деректер қорындағы Contribution моделімен байланысты жұмыс істейді. Әрбір үлес нақты бір қайырымдылық жобасына тіркеледі. Осы арқылы модератор қай жобаға кімнен қандай көмек ұсынылғанын көре алады. Сонымен қатар пайдаланушы қосымша хабарлама қалдыра алады, мысалы, “киімдерді өзім жеткізіп беремін” немесе “демалыс күні ерікті ретінде көмектесемін” деген сияқты ақпарат модератор үшін өте маңызды болуы мүмкін.

Үлес қосу бетінің мазмұны платформаның негізгі идеясымен тікелей байланысты. Бұл бет пайдаланушыға тек ақша аудару емес, әртүрлі көмек түрлерін ұсынуға болатынын көрсетеді. Мысалы, бір пайдаланушы заттай көмек ұсынса, екіншісі ерікті ретінде қатыса алады, ал үшінші адам кеңес беру арқылы көмектесуі мүмкін.

Сонымен қатар бұл модуль қайырымдылық процесінің ашықтығын арттырады. Себебі әрбір қаржылай үлес жобаның жалпы прогресіне әсер етеді, ал жиналған қаражат мөлшері пайдаланушыға көрініп тұрады. Бұл көмек көрсетушілердің сенімін арттырып, оларды белсенді қатысуға ынталандырады. Үлес қосу беті қарапайым, түсінікті және қолдануға ыңғайлы етіп жасалған, сондықтан пайдаланушы қажетті жобаны таңдап, бірнеше өрісті толтыру арқылы өз көмегін ұсына алады. Үлес қосу беті 6-суретте көрсетілген.

Мейірім Pro

Қайырымдылық платформасы

Жобалар

Көмек сұрау

Үлес қосу

Dashboard

Платформа

Admin

Анасы жұмыс іздеу кезеңінде. Балалардың күнделікті тамағы мен гигиена заттарын қамтамасыз ету үшін азық-түлік себеті жиналып жатыр.

Астана
Аймақ

2
Қаралым

6 күн
Мерзім

Жоба жаңалықтары

23.05.2026 13:30

Азық-түлік себеті дайындалуда

Ұн, май, күріш, балалар тағамы және гигиена заттары тізімге енгізілді.

Осы жобаға үлес қосу

Жоба:

Астанада жалғызбасты анаға азық-түлік себеті

Аты-жөніңіз:

Мысалы: Ерлан Нұрлан

Телефон нөмірі:

+7 701 000 00 00

Email:

name@example.com

Көмек түрі:

Қаржылай көмек

Сомма:

0

Атымды көрсетпеу: ☐

Қосымша хабарлама:

Қалай көмектесе алатыныңызды жазыңыз

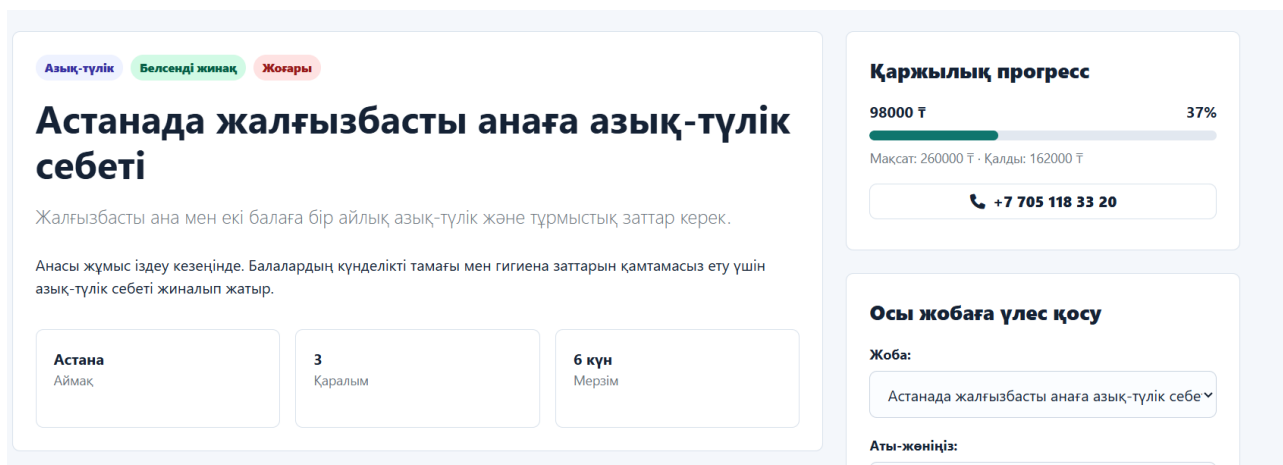
Активация Wi-Fi
Число активировано

Үлес қосу "Параметры"

6-сурет. Үлес қосу формасы

Жоба detail бетінде прогресс көрсеткіші маңызды рөл атқарады. Әр жоба үшін мақсатты сома және жиналған сома көрсетіледі. Осы екі мән негізінде жүйе пайыздық прогресті есептейді. Мысалы, егер мақсатты сома 500 000 теңге, ал жиналған сома 250 000 теңге болса, прогресс 50 пайыз болады. Бұл көрсеткіш пайдаланушыға жобаның қаншалықты орындалғанын бірден түсінуге мүмкіндік береді.

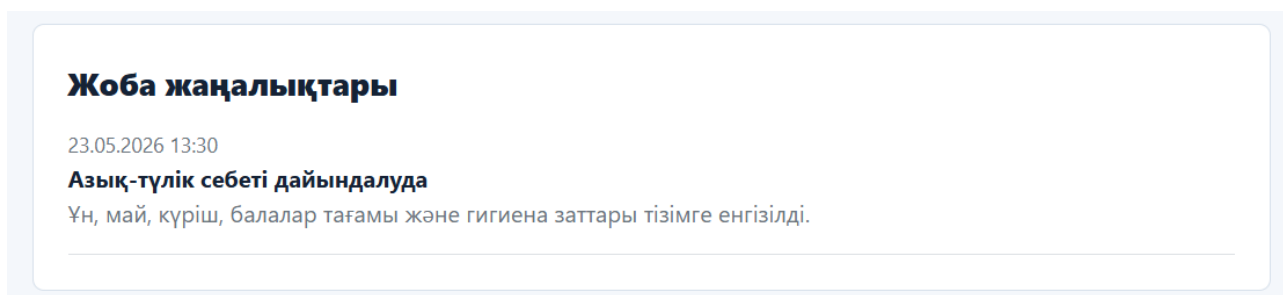
Прогресс бөлімі жобаға деген сенімді арттырады. Пайдаланушы қанша қаражат жиналғанын және қанша қаражат қалғанын көре алады. Сонымен қатар бұл көрсеткіш мотивациялық әсер береді, себебі мақсатқа аз ғана сома қалса, демеуші үлес қосуға көбірек ынталануы мүмкін. Жобаның қаржылық прогресс бөлімі 7-суретте көрсетілген.



7-сурет. Жоба прогресін көрсету бөлімі

Жоба жаңалықтары модулі де іске асырылды. Бұл модуль арқылы модератор әр жоба бойынша атқарылған жұмыстарды немесе маңызды өзгерістерді жариялай алады. Мысалы, “Алғашқы жеткізу жасалды”, “Дәрілер сатып алынды”, “Пеш шебері табылды” сияқты жаңалықтар жоба бетінде көрсетіледі. Мұндай мүмкіндік қайырымдылық процесінің ашықтығын арттырады.

Жаңалықтар бөлімі пайдаланушыға жоба бойынша нақты қозғалыс бар екенін көрсетеді. Егер демеуші бұрын үлес қосқан болса, ол кейін жоба бетіне кіріп, қандай нәтиже болғанын көре алады. Бұл қайырымдылық платформасы үшін өте маңызды, себебі ашық есеп беру пайдаланушылар сенімін күшейтеді. Жоба жаңалықтары бөлімі 8-суретте көрсетілген.

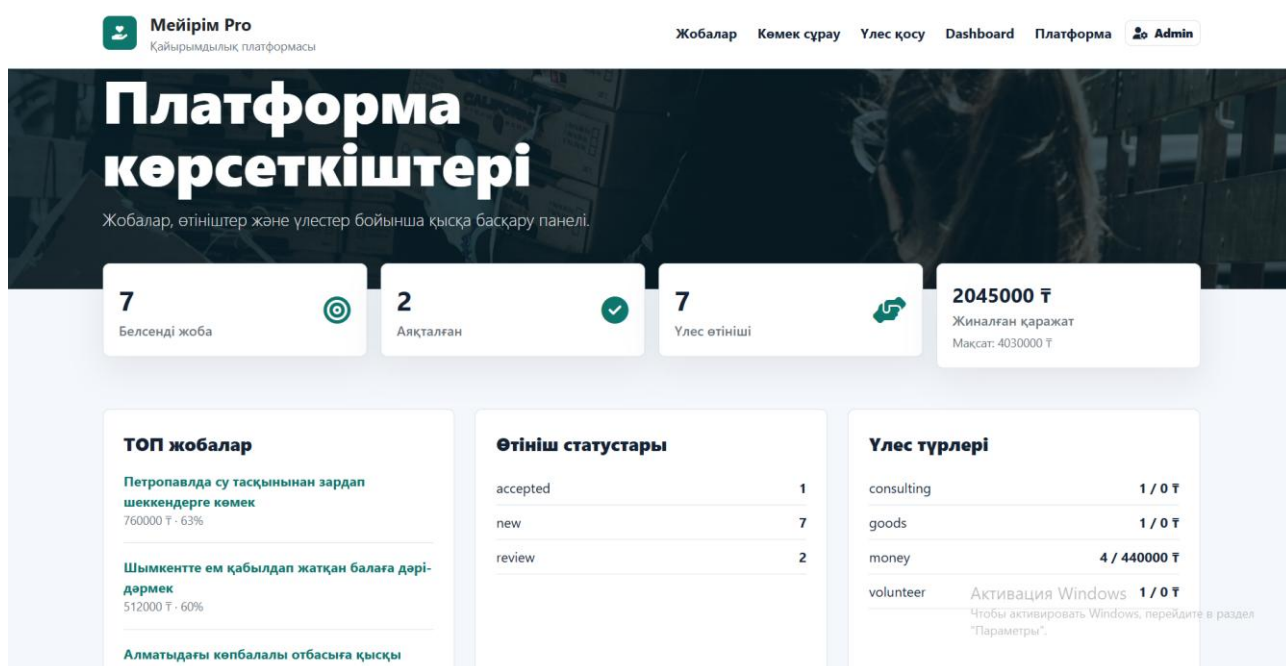


8-сурет. Жоба жаңалықтары бөлімі

Dashboard бөлімі платформаның жалпы жағдайын бақылауға арналған. Бұл бетте белсенді жобалар саны, аяқталған жобалар саны, үлес қосу өтініштері, қабылданған көмек өтініштері және жиналған қаражат сияқты көрсеткіштер беріледі. Сонымен қатар соңғы өтініштер мен соңғы үлестер кесте түрінде көрсетіледі.

Dashboard бетінің мазмұны модератор немесе жоба жетекшісі үшін пайдалы. Ол бір экранда платформаның ағымдағы жағдайын көре алады. Мысалы, қанша белсенді жоба бар, қанша өтініш қабылданды, қандай жобалар

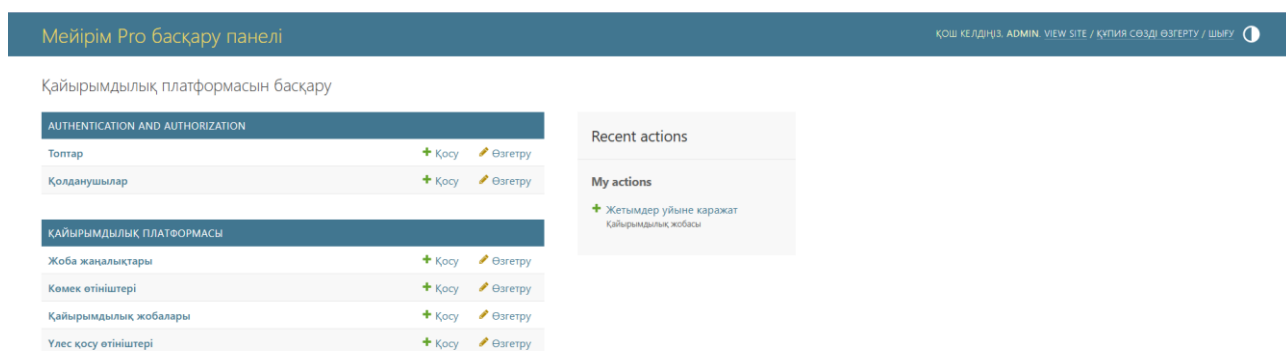
көп қаражат жинады, соңғы үлес қосушылар кімдер екенін бақылауға болады. Бұл басқару шешімдерін қабылдауға көмектеседі. Dashboard беті 9-суретте көрсетілген.



9-сурет. Dashboard және статистикалық көрсеткіштер беті

Әкімші панелі Django admin негізінде жасалды және қазақ тіліне бейімделді. Admin панельде қайырымдылық жобаларын, көмек өтініштерін, үлес қосу жазбаларын және жоба жаңалықтарын басқаруға болады. Әр модель үшін іздеу, сүзгілеу және өрістерді топтау мүмкіндіктері қосылды [12, 13, 14].

Admin панельдің қазақ тілінде болуы жүйені жергілікті қолданушылар үшін ыңғайлы етеді. Модератор “Қайырымдылық жобалары”, “Көмек өтініштері”, “Үлес қосу өтініштері” және “Жоба жаңалықтары” сияқты бөлімдерді түсінікті түрде көреді. Бұл әкімшілік жұмысты жеңілдетеді. Қазақ тілді admin панель 10-суретте көрсетілген.



10-сурет. Қазақ тілді әкімші панелі

Негізгі функционалдарды әзірлеу барысында жүйенің пайдаланушыға ыңғайлы болуына, деректердің дұрыс сақталуына және интерфейстің түсінікті болуына көңіл бөлінді. Әр модуль қайырымдылық процесінің нақты бір кезеңін автоматтандырады. Нәтижесінде платформа тек ақпараттық сайт емес, толыққанды басқару жүйесі ретінде қалыптасты.

3.3 Тестілеу және демо деректермен толтыру

Бағдарламалық өнімді әзірлеу аяқталғаннан кейін оның дұрыс жұмыс істейтінін тексеру қажет. Тестілеу жүйенің негізгі функционалдарының талаптарға сәйкес орындалуын анықтауға мүмкіндік береді. Қайырымдылық жобаларын басқаруға арналған веб-платформада тестілеу екі бағытта жүргізілді: автоматтандырылған тестілеу және қолмен тексеру.

Автоматтандырылған тестілеу Django TestCase құралдары арқылы орындалды. Бұл тесттер жүйенің негізгі беттері ашылатынын, жобалар каталогының сүзгілеу мүмкіндігі жұмыс істейтінін, көмекке өтініш беру формасы деректерді сақтайтынын және үлес қосу формасы жоба сомасын жаңартатынын тексереді.

Тестілеу барысында басты бет, жобалар каталогы, жоба detail беті, көмек сұрау беті, үлес қосу беті, dashboard және about беті тексерілді. Әр бет HTTP 200 статусымен ашылуы тиіс. Бұл беттердің серверлік жағынан дұрыс жұмыс істейтінін көрсетеді.

Жобалар каталогын тестілеу кезінде іздеу, категория, қала және маңыздылық бойынша сүзгілеу тексерілді. Мысалы, “киім” сөзі бойынша іздеу жүргізілгенде жүйе осы сөз кездесетін жобаларды ғана көрсетуі керек. Егер категория ретінде “clothes” таңдалса, тек киімге қатысты жобалар шығуы тиіс.

Көмекке өтініш беру формасын тестілеу кезінде пайдаланушы енгізген деректердің HelpRequest кестесіне сақталуы тексерілді. Форма сәтті жіберілгеннен кейін жүйе пайдаланушыға “Өтінішіңіз қабылданды” деген хабарлама көрсетуі тиіс. Бұл пайдаланушы әрекетінің сәтті аяқталғанын білдіреді [15].

Үлес қосу формасын тестілеу кезінде Contribution жазбасының сақталуы және жоба сомасының жаңаруы тексерілді. Мысалы, бастапқыда жиналған сома 45 000 теңге болса және пайдаланушы 50 000 теңге үлес қосса, жиналған сома 95 000 теңге болуы тиіс. Бұл логиканың дұрыс жұмыс істейтінін көрсетеді.

Автоматтандырылған тестілеу нәтижесінде негізгі функционалдар дұрыс жұмыс істейтіні анықталды. Тест нәтижелері 1-кестеде көрсетілген.

1-кесте. Тестілеу нәтижелері

Тест атауы	Күтілетін нәтиже	Нәтиже
Негізгі беттерді ашу	Барлық беттер 200 статусымен ашылады	Сәтті
Жобаларды сүзгілеу	Іздеу және сүзгі дұрыс жұмыс істейді	Сәтті
Көмек өтінішін сақтау	HelpRequest кестесіне жаңа жазба қосылады	Сәтті

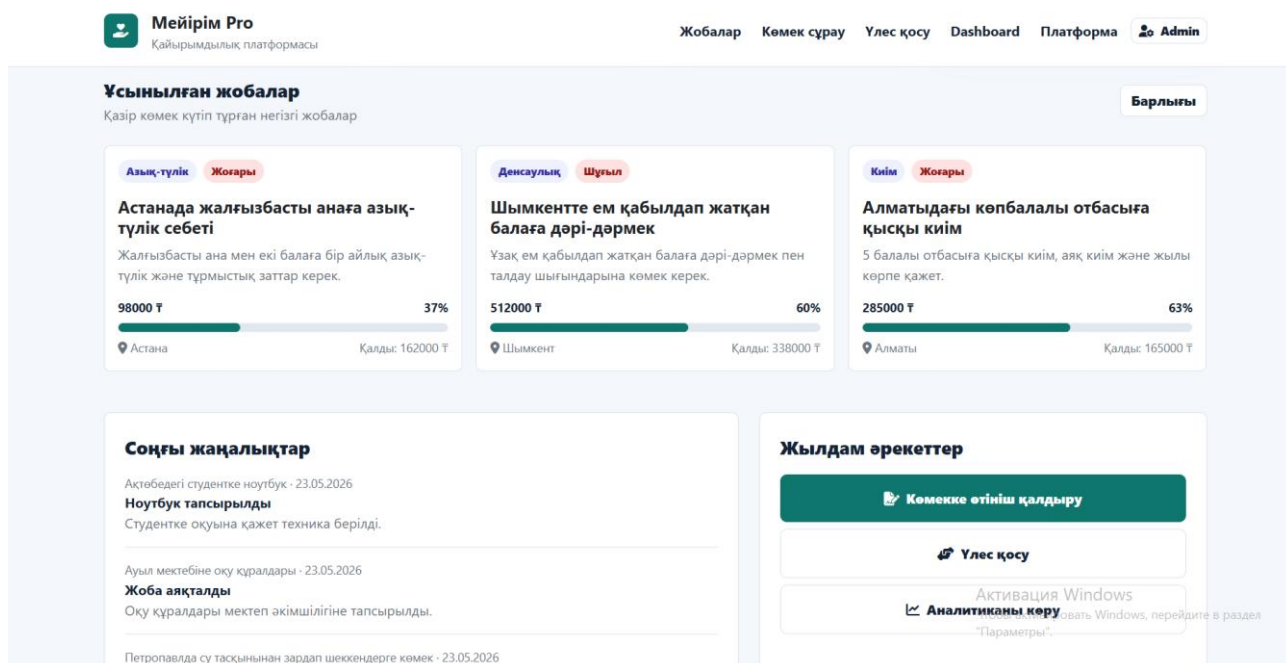
Үлес қосу	Contribution сақталып, жоба сомасы жаңарады	Сәтті
Admin панель	Қазақ тіліндегі басқару беті ашылады	Сәтті

Қолмен тексеру барысында веб-платформа браузер арқылы ашылып, негізгі беттердің визуалдық көрінісі қаралды. Басты бетте ұсынылған жобалар мен статистика дұрыс көрсетіледі. Жобалар каталогында карточкалар дұрыс орналасады. Жоба detail бетінде прогресс жолы, жиналған сома және үлес қосу формасы көрінеді. Dashboard бөлімінде жалпы статистика мен соңғы жазбалар көрсетіледі.

Демо деректермен толтыру үшін арнайы seed_demo management command жасалды. Бұл команда жүйеге бірнеше қайырымдылық жобасын, жоба жаңалықтарын, көмек өтініштерін және үлес қосу жазбаларын енгізеді. Демо деректер платформаның бос емес, нақты жұмыс істейтін жүйе сияқты көрінуіне мүмкіндік береді.

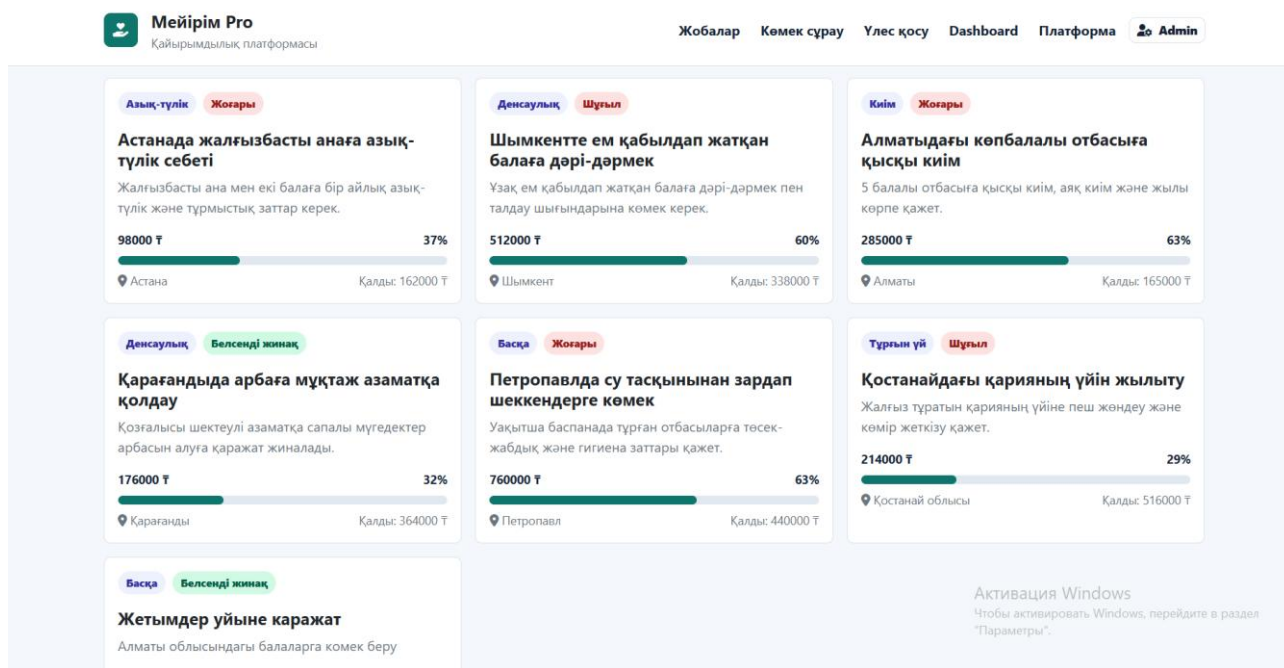
Демо деректер ретінде әртүрлі бағыттағы жобалар енгізілді. Мысалы, Алматыдағы көпбалалы отбасыға қысқы киім, Шымкентте ем қабылдап жатқан балаға дәрі-дәрмек, ауыл мектебіне оқу құралдары, Астанада жалғызбасты анаға азық-түлік себеті, Қостанайдағы қарияның үйін жылыту сияқты жобалар қосылды. Бұл жобалар платформаның әртүрлі категориялармен жұмыс істей алатынын көрсетеді.

Демо деректердің болуы қорғау кезінде өте пайдалы. Егер платформа бос болса, оның мүмкіндіктерін толық көрсету қиын болады. Ал демо деректер арқылы іздеу, сүзгілеу, прогресс көрсету, үлес қосу және dashboard сияқты функционалдарды нақты мысалмен көрсетуге болады. Демо деректермен толтырылған басты бет 11-суретте көрсетілген.



11-сурет. Демо деректермен толтырылған басты бет

Жобалар каталогында демо жобалар карточка түрінде көрсетіледі. Әр карточкада жоба атауы, категориясы, қаласы, жиналған сома және прогресс көрсеткіші бар. Бұл пайдаланушыға жобаларды салыстыруға және өзіне жақын жобаны таңдауға мүмкіндік береді. Демо жобалар каталогы 12-суретте көрсетілген.



12-сурет. Демо жобалар каталогы

Dashboard бетінде демо деректер негізінде статистика қалыптасады. Мысалы, белсенді жобалар саны, аяқталған жобалар саны, үлес өтініштері және жиналған қаражат көрсетіледі. Бұл бөлім модераторға платформадағы жалпы жағдайды түсінуге көмектеседі.

Тестілеу және демо деректермен толтыру нәтижесінде веб-платформаның негізгі функционалдары дұрыс жұмыс істейтіні анықталды. Жүйе қайырымдылық жобаларын басқару, көмек өтініштерін қабылдау, үлес қосу ұсыныстарын сақтау және статистикалық мәліметтерді көрсету міндеттерін орындайды. Сонымен қатар демо деректер платформаның практикалық қолданылуын көрсетуге мүмкіндік береді.

Осылайша, үшінші бөлімде веб-платформаның нақты іске асырылуы, негізгі модульдері, пайдаланушыға көрінетін беттері, admin панелі, тестілеу нәтижелері және демо деректермен толтыру процесі сипатталды. Бұл бөлім дипломдық жұмыстың практикалық бөлігін толық ашады және әзірленген жүйенің жұмысқа жарамды екенін дәлелдейді.

4 ЕҢБЕКТІ ҚОРҒАУ

4.1 Компьютермен жұмыс істеу қауіпсіздігі

Еңбекті қорғау – кез келген өндірістік немесе оқу-тәжірибелік жұмыстың маңызды бөлігі. Бағдарламалық өнім әзірлеу кезінде негізгі жұмыс құралы компьютер болғандықтан, еңбек қауіпсіздігі талаптарын сақтау ерекше мәнге ие. Компьютермен ұзақ уақыт жұмыс істеу адамның көру қабілетіне, жүйке жүйесіне, омыртқаға, қол бұлшықеттеріне және жалпы жұмыс қабілетіне әсер етуі мүмкін. Сондықтан бағдарламалық жүйені әзірлеу барысында жұмыс орнын дұрыс ұйымдастыру, қауіпсіздік ережелерін сақтау және денсаулыққа зиян келтіретін факторлардың алдын алу қажет.

Қайырымдылық жобаларын басқаруға арналған веб-платформаны әзірлеу барысында жұмыс негізінен компьютерде орындалды. Жоба кодын жазу, деректер қорын құру, интерфейсті дайындау, браузерде тексеру, құжаттама жазу және тестілеу әрекеттері ұзақ уақыт бойы экран алдында отыруды талап етеді. Осы себепті жұмыс кезінде санитарлық-гигиеналық, эргономикалық және ақпараттық қауіпсіздік талаптарын сақтау маңызды.

Бағдарламалық өнімді әзірлеу кезіндегі еңбек қауіпсіздігі тек физикалық қауіпсіздікпен шектелмейді. Қазіргі ақпараттық жүйелерде пайдаланушы деректерімен жұмыс істелетіндіктен, ақпараттық қауіпсіздік те еңбекті қорғау бөлімінің маңызды бағыты болып табылады. Әсіресе қайырымдылық платформасында пайдаланушылардың аты-жөні, телефон нөмірі, email мекенжайы және көмекке қатысты жеке мәліметтері сақталатындықтан, бұл деректерді қорғау қажет.

Осы бөлімде компьютермен жұмыс істеу қауіпсіздігі, жұмыс орнын ұйымдастыру талаптары және ақпараттық қауіпсіздік пен деректерді қорғау мәселелері қарастырылады.

Компьютермен жұмыс істеу кезінде адам ағзасына әртүрлі факторлар әсер етеді. Олардың қатарына көзге түсетін жүктеме, ұзақ уақыт бір қалыпта отыру, қол және иық бұлшықеттерінің шаршауы, жүйке жүйесіне түсетін салмақ, электр жабдықтарымен жұмыс істеу қаупі және жұмыс ортасының дұрыс ұйымдастырылмауы жатады. Егер бұл факторларға көңіл бөлінбесе, жұмыс өнімділігі төмендеп, денсаулыққа кері әсер етуі мүмкін.

Ең алдымен, компьютермен жұмыс істеген кезде көру қабілетін қорғау маңызды. Монитор экранына ұзақ қарау көздің шаршауына, құрғауына, бас ауруына және көрудің уақытша нашарлауына әкелуі мүмкін. Сондықтан монитор көзден шамамен 50–70 см қашықтықта орналасуы тиіс. Экранның жоғарғы бөлігі көз деңгейімен шамалас болуы керек. Бұл мойынға түсетін салмақты азайтады және көздің табиғи бағытта қарауына мүмкіндік береді.

Экран жарықтығы бөлме жарығымен үйлесімді болуы қажет. Егер экран тым жарық болса, көз тез шаршайды, ал тым күңгірт болса, мәтінді оқу қиындайды. Сонымен қатар мониторға күн сәулесі немесе шам жарығы тікелей

түспеуі керек, себебі шағылысу көзге қосымша жүктеме түсіреді. Жұмыс барысында экрандағы қаріп өлшемі оқуға ыңғайлы болуы тиіс.

Компьютермен ұзақ уақыт жұмыс істегенде үзіліс жасау міндетті. Әр 45–60 минут сайын 5–10 минуттық қысқа үзіліс жасаған дұрыс. Үзіліс кезінде көзді экраннан алыстатып, терезеге немесе алыс нүктеге қарау, жеңіл жаттығулар жасау, орнынан тұрып жүру ұсынылады. Бұл қан айналымын жақсартып, бұлшықеттердің шаршауын азайтады.

Компьютермен жұмыс істеу кезінде дене қалпы да маңызды. Арқа түзу, иық бос, аяқ еденге толық тиіп тұруы керек. Қол пернетақта мен тінтуірге ыңғайлы орналасуы тиіс. Егер орындық немесе үстел биіктігі дұрыс таңдалмаса, омыртқаға және мойынға артық салмақ түседі. Бұл ұзақ мерзімде арқа ауруына немесе бұлшықет шаршауына әкелуі мүмкін.

Электр қауіпсіздігі де компьютермен жұмыс істеу кезіндегі негізгі талаптардың бірі болып табылады. Компьютер, монитор, зарядтау құрылғысы және басқа да электр жабдықтары дұрыс қосылуы керек. Розеткаларға шамадан тыс жүктеме түсірмеу қажет. Электр сымдары зақымданбаған болуы тиіс және аяқ астында шашылып жатпауы керек. Сымдардың ретсіз орналасуы құлап қалу немесе электр жабдығының зақымдану қаупін арттырады.

Компьютер жанында сұйықтық ұстау қауіпті болуы мүмкін. Су, шай немесе басқа сұйықтық пернетақтаға немесе жүйелік блокқа төгілсе, қысқа тұйықталу пайда болуы мүмкін. Сондықтан жұмыс орнында сұйықтықты техникадан алыс қою қажет. Егер электр құрылғысы қызып кетсе немесе күйік иісі сезілсе, оны дереу тоқ көзінен ажыратып, маманға көрсету керек.

Бағдарламалық өнімді әзірлеу барысында психологиялық жүктеме де болуы мүмкін. Ұзақ уақыт код жазу, қателерді түзету, тестілеу және құжат дайындау адамды шаршатуы мүмкін. Сондықтан жұмыс уақытын дұрыс жоспарлау, тапсырмаларды кезеңге бөлу және демалыс режимін сақтау маңызды. Бұл жұмыс сапасын жақсартып қана қоймай, жүйке жүйесіне түсетін салмақты да азайтады.

Қайырымдылық жобаларын басқаруға арналған веб-платформаны әзірлеу барысында компьютермен жұмыс істеудің осы қауіпсіздік талаптары ескерілді. Жұмыс кезеңдері жоспарланып, код жазу, тестілеу және құжат дайындау әрекеттері ретімен орындалды. Бұл жұмыс өнімділігін сақтауға және денсаулыққа зиян келтірмеуге мүмкіндік берді.

4.2 Жұмыс орнын ұйымдастыру талаптары

Жұмыс орнын дұрыс ұйымдастыру еңбек қауіпсіздігін қамтамасыз етудің маңызды шарты болып табылады. Бағдарламашы немесе веб-жоба әзірлеуші көп уақытын компьютер алдында өткізетіндіктен, жұмыс орны ыңғайлы, жарық, қауіпсіз және эргономикалық талаптарға сай болуы керек. Дұрыс ұйымдастырылған жұмыс орны адамның шаршауын азайтады, еңбек өнімділігін арттырады және денсаулыққа кері әсер ететін факторлардың алдын алады.

Жұмыс үстелі жеткілікті кең болуы қажет. Үстелде монитор, пернетақта, тінтуір, дәптер немесе құжаттар еркін орналасуы тиіс. Қажетсіз заттардың көп болуы жұмысқа кедергі келтіреді және назарды бөледі. Сондықтан жұмыс үстелінде тек қажетті құралдар ғана болғаны дұрыс.

Монитор үстелдің ортасында орналасуы керек. Экран пайдаланушының көзіне тура бағытталуы тиіс. Егер монитор бір жаққа ығысқан болса, мойын мен омыртқаға қосымша салмақ түседі. Монитордың биіктігі көз деңгейіне сәйкес болуы қажет. Экран тым төмен немесе тым жоғары тұрса, мойын бұлшықеттері тез шаршайды.

Орындық эргономикалық талаптарға сай болуы тиіс. Орындықтың биіктігі реттелетін болғаны дұрыс. Арқа тіреуі омыртқаны ұстап тұруы керек. Пайдаланушы отырғанда тізе шамамен 90 градус бұрышта бүгіліп, аяқ еденге толық тиіп тұруы қажет. Егер аяқ еденге жетпесе, арнайы тіреуіш қолдануға болады.

Пернетақта мен тінтуір қолдың табиғи қалпында орналасуы керек. Қол білегі қатты бүгілмеуі тиіс. Пернетақта тым алыс немесе тым жақын болса, иық пен қол бұлшықеттеріне салмақ түседі. Тінтуірді қолданғанда қол үстел бетінде еркін қозғалуы қажет. Ұзақ уақыт тінтуірмен жұмыс істеу қол білегін шаршатуы мүмкін, сондықтан қысқа үзілістер жасау керек.

Жұмыс бөлмесінің жарықтануы маңызды рөл атқарады. Табиғи жарық болғаны жақсы, бірақ күн сәулесі экранға тікелей түспеуі тиіс. Жасанды жарық біркелкі болуы керек. Өте әлсіз жарықта жұмыс істеу көзді шаршатады, ал тым қатты жарық экрандағы мәтінді көруді қиындатады. Ең тиімдісі – бөлмедегі жалпы жарық пен экран жарықтығының үйлесімді болуы.

Бөлменің микроклиматы да еңбек өнімділігіне әсер етеді. Бөлме тым ыстық немесе тым суық болмауы керек. Желдету жүйесі дұрыс жұмыс істеуі қажет. Ұзақ уақыт жабық бөлмеде отыру адамның шаршауын арттырады, сондықтан бөлмені жиі желдетіп отырған дұрыс. Ауа температурасы шамамен 20–24 градус аралығында болғаны ыңғайлы.

Жұмыс орнында шу деңгейі төмен болуы қажет. Артық шу адамның назарын бөледі және жұмыс сапасын төмендетеді. Бағдарламалық өнім әзірлеу кезінде ойлану, код құрылымын түсіну және қателерді табу үшін тыныш орта қажет. Егер сыртқы шу көп болса, жұмыс өнімділігі төмендеуі мүмкін.

Құжаттармен жұмыс істеген кезде оларды монитор жанына немесе арнайы құжат ұстағышқа орналастырған дұрыс. Бұл көз бен мойын қозғалысын азайтады. Егер құжаттар үстелдің шетінде немесе төменде жатса, пайдаланушы жиі еңкейіп, омыртқаға артық салмақ түсіреді.

Жұмыс орнында қауіпсіздік тәртібі сақталуы тиіс. Электр сымдары реттелген болуы керек, жолда жатпауы қажет. Құрылғылардың желдеткіш тесіктері жабылмауы тиіс, себебі бұл техниканың қызып кетуіне әкелуі мүмкін. Компьютер жабдықтарын шаңнан тазалап отыру да маңызды, өйткені шаң құрылғының қызып кетуіне және жұмыс өнімділігінің төмендеуіне әсер етеді.

Қайырымдылық веб-платформасын әзірлеу барысында жұмыс орны осы талаптарға сай ұйымдастырылды. Компьютер, браузер, мәтіндік редактор және

серверлік орта бір жұмыс аймағында қолданылды. Жоба файлдары реттелген құрылымда сақталып, код жазу мен тестілеу кезеңдері жүйелі түрде жүргізілді. Бұл жұмыс процесінің тиімді болуына және қателерді азайтуға мүмкіндік берді.

Дұрыс ұйымдастырылған жұмыс орны бағдарламалық өнім сапасына да әсер етеді. Егер жұмыс ортасы ыңғайлы болса, әзірлеуші ұзақ уақыт бойы өнімді жұмыс істей алады, қателерді тез табады және жүйені сапалы іске асырады. Сондықтан еңбек қауіпсіздігі мен эргономика талаптарын сақтау бағдарламалық жобаның сәтті орындалуына тікелей ықпал етеді.

4.3 Ақпараттық қауіпсіздік және деректерді қорғау

Ақпараттық қауіпсіздік – қазіргі веб-жүйелер үшін ең маңызды талаптардың бірі. Қайырымдылық жобаларын басқаруға арналған веб-платформада пайдаланушылардың жеке деректері сақталады. Оларға аты-жөні, телефон нөмірі, email мекенжайы, қала, көмекке қатысты хабарлама және үлес қосу туралы ақпарат жатады. Мұндай деректердің қауіпсіз сақталуы пайдаланушылардың сенімі үшін өте маңызды.

Веб-платформада ақпараттық қауіпсіздікті қамтамасыз ету бірнеше бағыт бойынша қарастырылады. Бірінші бағыт – пайдаланушы енгізетін деректерді дұрыс қабылдау және тексеру. Екінші бағыт – деректер қорындағы ақпаратты рұқсатсыз өзгертуден қорғау. Үшінші бағыт – әкімші панельге тек рұқсаты бар адамның кіруін қамтамасыз ету. Төртінші бағыт – жүйедегі деректердің тұтастығын сақтау.

Жобада пайдаланушыдан дерек қабылдау үшін Django Forms қолданылды. Формалар енгізілген мәліметтерді модель өрістеріне сәйкес тексереді. Мысалы, email өрісіне email форматына сәйкес келмейтін мән енгізілсе, жүйе оны қабылдамауы мүмкін. Бұл деректердің сапасын арттырады және қате ақпараттың сақталуын азайтады.

Django фреймворкінде CSRF қорғанысы қарастырылған. CSRF, яғни Cross-Site Request Forgery, қолданушы атынан рұқсатсыз сұраныс жіберу шабуылдарының алдын алуға арналған. Жүйедегі формаларда CSRF token қолданылады. Бұл token әр форма жіберілген кезде тексеріледі. Егер token дұрыс болмаса, сұраныс қабылданбайды. Бұл веб-платформаның қауіпсіздігін арттырады.

Әкімші панельге кіру тек логин және құпиясөз арқылы жүзеге асырылады. Django admin панелі пайдаланушыны аутентификациядан өткізеді. Құпиясөздер ашық мәтін түрінде сақталмайды, олар арнайы hashing механизмі арқылы қорғалады. Бұл құпиясөздердің қауіпсіздігін қамтамасыз етеді.

Admin панель арқылы тек рұқсаты бар модератор немесе әкімші ғана деректерді өзгерте алады. Бұл өте маңызды, себебі қайырымдылық жобаларына қатысты ақпараттың дұрыс болуы пайдаланушылар сеніміне әсер етеді. Егер кез келген адам жоба сомасын немесе мәртебесін өзгерте алса, жүйенің сенімділігі төмендейді. Сондықтан басқару мүмкіндіктері тек әкімші панель арқылы шектеледі.

Деректер қорындағы ақпараттың тұтастығын сақтау үшін модельдер арасындағы байланыстар дұрыс ұйымдастырылды. Мысалы, Contribution моделі нақты CharityProject жобасына байланысады. Бұл әр үлес қосу жазбасының қай жобаға тиесілі екенін анық көрсетеді. ProjectUpdate моделі де нақты жобаға байланысады, сондықтан жаңалықтар жүйесіз сақталмайды.

Жүйеде қаржылай үлес қосылған жағдайда жоба сомасы автоматты түрде жаңартылады. Бұл кезде қайталама қосу қателігін болдырмау үшін `applied_to_total` өрісі қолданылады. Егер үлес сомасы бұрын жобаға қосылған болса, оны қайта қосуға жол бермеу қажет. Бұл деректердің дұрыстығын сақтауға көмектеседі. Пайдаланушы деректерін қорғауда құпиялылық қағидалары да маңызды. Көмек сұраушының жеке жағдайы, байланыс деректері және хабарламасы ашық түрде жарияланбауы тиіс. Мұндай ақпаратты тек модератор көруі керек. Ал сайтта жарияланатын жобаларда тек қажет және тексерілген ақпарат көрсетілуі тиіс. Үлес қосушы үшін де құпиялылық мүмкіндігі қарастырылған. Contribution моделінде `is_anonymous` өрісі бар. Егер пайдаланушы өз аты-жөнін көрсеткісі келмесе, анонимді түрде көмектесе алады. Бұл кейбір демеушілер үшін маңызды болуы мүмкін және платформаның қолдану ыңғайлылығын арттырады.

Ақпараттық қауіпсіздіктің тағы бір маңызды бөлігі – резервтік көшіру. Бұл дипломдық жұмыста негізгі деректер SQLite файлында сақталады. Болашақта жүйе нақты қолданысқа енгізілсе, деректер қорын тұрақты түрде резервтік көшіру қажет. Резервтік көшірме деректер жоғалған жағдайда жүйені қалпына келтіруге мүмкіндік береді.

Сонымен қатар нақты өндірістік ортада HTTPS протоколын қолдану қажет. HTTPS пайдаланушы браузері мен сервер арасындағы деректерді шифрлап жеткізеді. Бұл әсіресе телефон, email және жеке хабарлама сияқты мәліметтер жіберілген кезде маңызды. Оқу жобасында локалды сервер қолданылғанымен, болашақта интернетке жариялау кезінде HTTPS міндетті түрде бапталуы тиіс. Жүйені қорғау үшін бағдарламалық қамтылымды уақытылы жаңартып отыру да қажет. Django, Python және басқа кітапханалардың жаңа нұсқаларында қауіпсіздік қателері түзетілуі мүмкін. Сондықтан production ортада тәуелділіктерді бақылап, қажет болған жағдайда жаңарту ұсынылады.

Ақпараттық қауіпсіздік тек техникалық құралдармен ғана шектелмейді. Пайдаланушылар мен модераторлар да қауіпсіздік ережелерін сақтауы тиіс. Мысалы, admin панель құпиясөзін бөгде адамдарға бермеу, әлсіз құпиясөз қолданбау, қоғамдық компьютерден шыққан соң жүйеден шығу және күмәнді файлдарды жүктемеу қажет. Қайырымдылық жобаларын басқаруға арналған веб-платформада ақпараттық қауіпсіздік талаптары бастапқы деңгейде ескерілді. Django формалары, CSRF қорғанысы, admin authentication, модельдер арасындағы байланыстар және қазақ тілді басқару панелі жүйенің қауіпсіз әрі түсінікті жұмыс істеуіне мүмкіндік береді.

Қорытындылай келе, ақпараттық қауіпсіздік пен деректерді қорғау қайырымдылық платформасының сенімділігі үшін маңызды фактор болып

табылады. Пайдаланушылар өз деректерінің қауіпсіз сақталатынына сенген жағдайда ғана платформаны белсенді қолданады. Сондықтан деректерді қорғау, құпиялылықты сақтау және жүйеге рұқсатсыз қол жеткізудің алдын алу веб-платформаның негізгі талаптарының бірі болып саналады.

5 ЭКОНОМИКАЛЫҚ БӨЛІМ

5.1 Жобаның экономикалық және әлеуметтік мәні

Экономикалық бөлімде әзірленген веб-платформаның экономикалық және әлеуметтік маңызы, жобаны жасауға кеткен шығындар және оны қолданудан күтілетін тиімділік қарастырылады. Бағдарламалық өнімді әзірлеу тек техникалық тұрғыдан ғана емес, экономикалық жағынан да негізделуі тиіс. Себебі кез келген ақпараттық жүйе белгілі бір ресурстарды қажет етеді: уақыт, еңбек, техника, интернет, электр энергиясы және бағдарламалық құралдар.

Қайырымдылық жобаларын басқаруға арналған веб-платформа әлеуметтік бағыттағы ақпараттық жүйе болып табылады. Оның басты мақсаты – табыс табу емес, қайырымдылық процестерін тиімді ұйымдастыру, көмекке мұқтаж адамдар мен көмек көрсетушілер арасындағы байланысты жеңілдету және әлеуметтік жобалардың ашықтығын арттыру. Дегенмен мұндай жүйенің экономикалық маңызы да бар, себебі ол адам еңбегін үнемдейді, ақпарат өңдеу уақытын қысқартады және ұйым жұмысының тиімділігін арттырады.

Жобаның экономикалық тиімділігін бағалау кезінде оны әзірлеуге кеткен шығындар, жүйені қолданудан болатын уақыт үнемі, әкімшілік жұмыстың жеңілдеуі және болашақта платформаны кеңейту мүмкіндіктері ескеріледі. Бұл бөлімде веб-платформаның әлеуметтік және экономикалық мәні, әзірлеу шығындары және даму мүмкіндіктері сипатталады.

Қайырымдылық жобаларын басқаруға арналған веб-платформаның экономикалық мәні оның қайырымдылық ұйымдары мен еріктілер топтарының жұмысын оңтайландыруымен байланысты. Дәстүрлі жағдайда көмекке өтініштерді қабылдау, жобаларды тізімдеу, демеушілермен байланысу, жиналған соманы бақылау және нәтиже туралы ақпарат беру қолмен немесе әртүрлі жеке құралдар арқылы жүргізіледі. Бұл көп уақыт алады және ақпараттың шашыраңқы болуына әкеледі.

Веб-платформа осы процестерді бір ортаға біріктіреді. Жүйеде көмек өтініштері бөлек сақталады, қайырымдылық жобалары құрылымдалады, үлес қосу ұсыныстары тіркеледі және жоба прогресі автоматты түрде көрсетіледі. Нәтижесінде модератор немесе ұйым қызметкері бірнеше бөлек файлмен, чатпен немесе кестемен жұмыс істемей, барлық ақпаратты бір жүйеден көре алады.

Экономикалық тұрғыдан алғанда, мұндай жүйе уақыт шығынын азайтады. Мысалы, егер өтініштер қолмен қабылданса, әр өтінішті жазу, өңдеу, тексеру және бөлек тізімге енгізу үшін көп уақыт қажет болады. Ал веб-платформада пайдаланушы форманы өзі толтырады, ал деректер автоматты түрде деректер қорына сақталады. Бұл модератордың жұмысын жеңілдетеді және қателіктер санын азайтады.

Платформаның тағы бір экономикалық артықшылығы – ашық технологияларға негізделуі. Жоба Python, Django, SQLite, HTML, CSS және Bootstrap сияқты тегін әрі кең таралған құралдар арқылы жасалды. Бұл

лицензиялық шығындарды азайтады. Сонымен қатар бұл технологиялар бойынша оқу материалдары көп, сондықтан жүйені әрі қарай дамыту және сүйемелдеу жеңіл болады.

Әлеуметтік мәні жағынан бұл платформа көмекке мұқтаж адамдар үшін қолжетімді құрал болып табылады. Олар арнайы мекемеге бармай-ақ, онлайн түрде өтініш қалдыра алады. Бұл әсіресе уақыты шектеулі, денсаулығы нашар немесе алыс аймақта тұратын адамдар үшін ыңғайлы. Платформа арқылы олардың өтініші жүйеде тіркеліп, модератор тарапынан қаралуы мүмкін.

Көмек көрсетушілер үшін де платформаның әлеуметтік маңызы жоғары. Демеуші немесе ерікті жобалар тізімін көріп, өзіне жақын бағытты таңдай алады. Мысалы, бір адам денсаулық саласындағы жобаларға көмектескісі келсе, екінші адам азық-түлік немесе киім бағытындағы жобаларды таңдай алады. Мұндай таңдау мүмкіндігі қайырымдылыққа қатысу белсенділігін арттырады.

Платформа қайырымдылық процесінің ашықтығын арттырады. Әр жоба бойынша мақсатты сома, жиналған сома және прогресс көрсеткіші беріледі. Бұл көмек көрсетушілерге қаражаттың жиналу барысын бақылауға мүмкіндік береді. Сонымен қатар жоба жаңалықтары арқылы атқарылған жұмыстарды көрсетуге болады. Ашықтық сенімді күшейтеді, ал сенім қайырымдылық саласындағы ең маңызды факторлардың бірі болып табылады.

Жобаның әлеуметтік тиімділігі еріктілер жұмысын үйлестіруде де көрінеді. Платформада қаржылай көмекпен қатар заттай көмек, ерікті болу және кеңес беру сияқты көмек түрлері қарастырылған. Бұл қайырымдылықты тек ақша аудару ретінде емес, әртүрлі әлеуметтік қолдау түрлерінің жиынтығы ретінде көрсетуге мүмкіндік береді.

Қорытындылай келе, жобаның экономикалық мәні – уақытты үнемдеу, ақпаратты жүйелеу, қол еңбегін азайту және ұйым жұмысын тиімді ету. Ал әлеуметтік мәні – көмекке мұқтаж адамдарға қолжетімді өтініш беру құралын ұсыну, демеушілер мен еріктілерді жобалармен байланыстыру және қайырымдылық процесінің ашықтығын қамтамасыз ету.

5.2 Жобаны әзірлеу шығындарын есептеу

Бағдарламалық өнімді әзірлеу белгілі бір шығындарды қажет етеді. Бұл дипломдық жобада шығындар оқу жобасы деңгейінде есептелді. Жобаны жасау кезінде негізгі шығын түрлеріне компьютер амортизациясы, интернет төлемі, электр энергиясы, бағдарламалық құралдарды пайдалану және әзірлеушінің еңбек уақыты жатады.

Жоба ашық және тегін технологиялар негізінде жасалғандықтан, бағдарламалық лицензияларға қосымша шығын қарастырылмайды. Python, Django, SQLite, Bootstrap және Visual Studio Code сияқты құралдарды тегін қолдануға болады. Бұл жобаның жалпы құнын төмендетеді.

Есептеу үшін жобаны әзірлеу мерзімі 2 ай деп алынды. Бұл уақыт ішінде жүйенің теориялық негіздері қарастырылып, деректер қоры жобаланды, Django

жобасы құрылды, негізгі беттер жасалды, admin панель бапталды, демо деректер енгізілді және тестілеу жүргізілді.

Компьютер амортизациясын есептеу үшін компьютердің орташа құны 350 000 теңге деп алынды. Компьютердің пайдалану мерзімі 5 жыл немесе 60 ай деп есептеледі.

Айлық амортизация:

$$350\,000 / 60 = 5\,833 \text{ теңге}$$

Жоба 2 ай әзірленгендіктен:

$$5\,833 \times 2 = 11\,666 \text{ теңге}$$

Интернет шығыны айына 7 000 теңге деп алынды. Екі айға:

$$7\,000 \times 2 = 14\,000 \text{ теңге}$$

Электр энергиясы шығынын есептеу үшін компьютердің орташа қуат тұтынуы және жұмыс уақыты ескерілді. Егер компьютер күніне орта есеппен 4 сағат қолданылса, ал жоба бойынша жұмыс 2 ай бойы жүргізілсе, электр энергиясына кететін шығын шамамен 1 200 теңге деп алынды.

Бағдарламалық құралдарға лицензиялық шығын жоқ, себебі қолданылған технологиялар тегін. Дегенмен қосымша техникалық және ұйымдастырушылық шығындар ретінде 10 000 теңге қарастырылды. Бұл шығындарға кеңсе тауарлары, интернеттен материал іздеу, құжат дайындау және басқа ұсақ қажеттіліктер кіреді.

Жобаны әзірлеуге кеткен шығындар төмендегі кестеде көрсетілген.

2-кесте. Жобаны әзірлеуге кеткен шығындар

№	Шығын атауы	Есептеу жолы	Сума
1	Компьютер амортизациясы	$350\,000 / 60 \times 2 \text{ ай}$	11 666 тг
2	Интернет шығыны	$7\,000 \times 2 \text{ ай}$	14 000 тг
3	Электр энергиясы	Шамамен есептеу	1 200 тг
4	Бағдарламалық құралдар	Ашық технологиялар	0 тг
5	Қосымша шығындар	Кеңсе және техникалық қажеттілік	10 000 тг
Барлығы			36 866 тг

Кестеден көріп отырғанымыздай, жобаны әзірлеуге кеткен жалпы шығын шамамен 36 866 теңгені құрайды. Бұл көрсеткіш оқу жобасы үшін қолайлы, себебі негізгі бағдарламалық құралдар тегін қолданылды.

Егер бұл платформаны нақты ұйымға бейімдеп, өндірістік ортаға енгізу қажет болса, қосымша шығындар пайда болуы мүмкін. Мысалы, домен сатып алу, хостинг төлемі, SSL сертификаты, серверді баптау, техникалық қолдау және жүйені сүйемелдеу шығындары қарастырылуы мүмкін. Бірақ бастапқы нұсқаны әзірлеу үшін жоғарыда көрсетілген шығындар жеткілікті.

Жобаның экономикалық жағынан тиімді болуы оның төмен шығынмен жасалуында ғана емес, сонымен қатар болашақта ұйым жұмысының уақытын

үнемдеуінде. Егер платформа өтініштерді қабылдау мен жобаларды басқару уақытын қысқартса, оның экономикалық әсері біртіндеп арта түседі.

5.3 Жобаның тиімділігі және даму мүмкіндіктері

Жобаның тиімділігі бірнеше бағыт бойынша бағаланады: уақыт үнемі, ақпараттың жүйеленуі, пайдаланушыға ыңғайлылық, ашықтық, модератор жұмысының жеңілдеуі және болашақта кеңейту мүмкіндігі. Қайырымдылық платформасының негізгі тиімділігі – бұрын қолмен орындалатын процестерді автоматтандыру.

Дәстүрлі жағдайда көмек өтініштері әртүрлі мессенджерлерде, әлеуметтік желілерде немесе қағаз түрінде сақталуы мүмкін. Бұл ақпаратты кейін іздеу, сұрыптау немесе талдау қиын. Ал веб-платформада барлық өтініштер бір деректер қорында сақталады. Модератор өтініштерді мәртебесі, көмек түрі немесе басымдық деңгейі бойынша қарай алады. Бұл жұмыс уақытын қысқартады.

Мысалы, қолмен бір өтінішті қабылдау және кестеге енгізу орта есеппен 20 минут уақыт алуы мүмкін. Ал веб-платформада пайдаланушы форманы өзі толтырады, ал модератор тек оны қарап шығады. Бұл процесс шамамен 8 минутқа дейін қысқаруы мүмкін.

Бір өтініш бойынша уақыт үнемі:

$20 \text{ минут} - 8 \text{ минут} = 12 \text{ минут}$

Егер айына 100 өтініш өңделсе:

$100 \times 12 \text{ минут} = 1200 \text{ минут}$

$1200 \text{ минут} = 20 \text{ сағат}$

Яғни платформа айына шамамен 20 сағат жұмыс уақытын үнемдеуі мүмкін. Егер модератордың бір сағат жұмысы шартты түрде 2 000 теңге деп есептелсе:

$20 \times 2\,000 = 40\,000 \text{ теңге}$

Демек, жүйе айына шамамен 40 000 теңге көлемінде еңбек уақытын үнемдеуге мүмкіндік береді. Бұл көрсеткіш жобаны әзірлеуге кеткен бастапқы шығыннан жоғары. Сондықтан платформа қысқа уақыт ішінде өзін ақтай алады деп айтуға болады.

Жобаның тиімділігі тек қаржылай есеппен шектелмейді. Әлеуметтік тиімділік те маңызды. Платформа көмекке мұқтаж адамдарға өтініш беруді жеңілдетеді, көмек көрсетушілерге сенімді жобаларды таңдауға мүмкіндік береді, ал модераторларға барлық ақпаратты бір жерде басқаруға жағдай жасайды. Бұл қайырымдылық процесінің сапасын арттырады.

Жүйенің тағы бір тиімділігі – ашықтық. Әр жоба бойынша мақсатты сома, жиналған сома және прогресс көрсеткіші көрсетіледі. Бұл демеушілердің сенімін арттырады. Егер демеуші өз көмегінің нақты жобаға қосылғанын көрсе, ол болашақта да платформа арқылы көмек көрсетуге дайын болуы мүмкін.

Dashboard бөлімі де тиімділікті арттырады. Бұл бөлімде белсенді жобалар саны, аяқталған жобалар, үлес қосу жазбалары, көмек өтініштері және

жиналған қаражат туралы мәліметтер көрсетіледі. Мұндай статистика ұйым басшылығына немесе модераторға платформадағы жалпы жағдайды жылдам бағалауға мүмкіндік береді.

Жобаны болашақта дамыту мүмкіндіктері кең. Бірінші бағыт – онлайн төлем жүйесін қосу. Егер платформаға Kaspi, банк картасы немесе басқа төлем жүйелері біріктірілсе, пайдаланушы үлес қосу процесін толық онлайн аяқтай алады. Бұл жүйенің практикалық маңызын арттырады.

Екінші бағыт – пайдаланушының жеке кабинетін жасау. Жеке кабинет арқылы көмек көрсетуші өзінің бұрынғы үлестерін көре алады, ал көмек сұраушы өз өтінішінің мәртебесін бақылай алады. Бұл пайдаланушы тәжірибесін жақсартады.

Үшінші бағыт – хабарландыру жүйесін енгізу. Мысалы, өтініш қабылданғанда, жоба мәртебесі өзгергенде немесе жоба аяқталғанда пайдаланушыға email немесе SMS хабарлама жіберуге болады. Бұл платформа мен пайдаланушы арасындағы байланысты күшейтеді.

Төртінші бағыт – файл немесе сурет жүктеу мүмкіндігі. Қайырымдылық жобаларында кейде растайтын құжаттар, фотосуреттер немесе есеп файлдары қажет болуы мүмкін. Бұл мүмкіндік модераторға өтінішті тексеруге және жоба нәтижесін көрсетуге көмектеседі.

Бесінші бағыт – PDF есеп генерациясы. Қайырымдылық ұйымдары үшін ай сайын немесе жоба аяқталғаннан кейін есеп шығару маңызды. Егер жүйе автоматты түрде PDF есеп жасай алса, ұйымның есеп беру жұмысы жеңілдейді.

Алтыншы бағыт – деректер қорын кеңейту және production ортаға көшіру. Қазіргі оқу жобасында SQLite қолданылды. Ал нақты қолданыста PostgreSQL сияқты қуатты деректер қорына көшу ұсынылады. Сонымен қатар жүйені серверге орналастыру, HTTPS қосу және резервтік көшіру механизмін баптау қажет.

Жобаның даму мүмкіндіктері оның архитектурасының дұрыс құрылғанын көрсетеді. Django негізіндегі құрылым жаңа модульдерді біртіндеп қосуға мүмкіндік береді. Мысалы, жаңа модель, жаңа форма немесе жаңа бет қосу арқылы платформаның функционалын кеңейтуге болады.

Қорытындылай келе, әзірленген веб-платформа экономикалық және әлеуметтік тұрғыдан тиімді. Ол өтініштерді өңдеу уақытын қысқартады, ақпаратты жүйелейді, қайырымдылық процесінің ашықтығын арттырады және модератор жұмысын жеңілдетеді. Болашақта қосымша модульдер енгізу арқылы платформаны нақты қайырымдылық ұйымдарына бейімдеуге болады.

ҚОРЫТЫНДЫ

Дипломдық жұмыс барысында «Қайырымдылық жобаларын басқаруға арналған веб-платформа» тақырыбы бойынша ақпараттық жүйе әзірленді. Жұмыстың негізгі мақсаты – қайырымдылық жобаларын цифрлық ортада басқару, көмекке мұқтаж адамдар мен көмек көрсетушілер арасындағы байланысты жеңілдету және жоба туралы ақпаратты ашық көрсету болды.

Жұмыстың теориялық бөлімінде қайырымдылық қызметін цифрландырудың маңызы, қайырымдылық жобаларын басқару ерекшеліктері және веб-платформаға қойылатын негізгі талаптар қарастырылды. Қайырымдылық саласында ақпараттың ашық болуы, жиналған қаражаттың көрінуі, өтініштердің жүйелі сақталуы және пайдаланушылар сенімінің қалыптасуы маңызды екені анықталды.

Жобалау бөлімінде жүйенің мақсаты, міндеттері, пайдаланушы рөлдері және деректер қорының құрылымы анықталды. Платформада көмек сұраушы, көмек көрсетуші және модератор рөлдері қарастырылды. Деректер қорында қайырымдылық жобалары, көмек өтініштері, үлес қосу жазбалары және жоба жаңалықтары үшін жеке модельдер жасалды.

Практикалық бөлімде Django фреймворкі негізінде көп бетті веб-платформа іске асырылды. Жүйеде басты бет, жобалар каталогы, жоба туралы толық ақпарат беті, көмекке өтініш беру беті, үлес қосу беті, dashboard және қазақ тілді әкімші панелі жасалды. Пайдаланушы жобаларды қарап, сүзгілей алады, көмекке өтініш қалдырады және нақты жобаға үлес қоса алады.

Жүйеде қаржылай үлес қосылған жағдайда жобаға жиналған сома автоматты түрде жаңарып, прогресс көрсеткіші өзгереді. Бұл қайырымдылық процесінің ашықтығын арттырады. Сонымен қатар жоба жаңалықтары арқылы атқарылған жұмыстарды көрсету мүмкіндігі қарастырылған.

Тестілеу нәтижесінде негізгі беттердің дұрыс ашылатыны, көмек өтінішінің сақталатыны, үлес қосу жазбасының тіркелетіні және жоба сомасының жаңаратыны анықталды. Жоба демо деректермен толтырылып, оның нақты жұмыс істеу мүмкіндіктері көрсетілді.

Экономикалық тұрғыдан платформа өтініштерді өңдеуге кететін уақытты қысқартады, ақпаратты жүйелейді және модератор жұмысын жеңілдетеді. Әлеуметтік тұрғыдан жүйе көмекке мұқтаж адамдарға өтініш беруді жеңілдетіп, демеушілер мен еріктілердің қайырымдылық жобаларына қатысуына жағдай жасайды.

Қорытындылай келе, дипломдық жұмыста қойылған мақсат орындалды. Әзірленген веб-платформа қайырымдылық жобаларын басқаруға, көмек өтініштерін қабылдауға, үлес қосу ұсыныстарын тіркеуге және жоба барысын бақылауға мүмкіндік береді. Болашақта жүйені онлайн төлем, жеке кабинет, хабарландыру және есеп беру модульдерімен толықтыруға болады.

ПАЙДАЛАНЫЛҒАН ӘДЕБИЕТТЕР

- 1 Django Software Foundation. Django Documentation. URL: <https://docs.djangoproject.com/>
- 2 Python Software Foundation. Python 3 Documentation. URL: <https://docs.python.org/3/>
- 3 Bootstrap Documentation. URL: <https://getbootstrap.com/docs/>
- 4 SQLite Documentation. URL: <https://www.sqlite.org/docs.html>
- 5 Mozilla Developer Network. HTML: HyperText Markup Language. URL: <https://developer.mozilla.org/>
- 6 Mozilla Developer Network. CSS: Cascading Style Sheets. URL: <https://developer.mozilla.org/>
- 7 OWASP Foundation. Web Security Testing Guide. URL: <https://owasp.org/>
- 8 Қазақстан Республикасының «Дербес деректер және оларды қорғау туралы» Заңы.
- 9 Sommerville I. Software Engineering. Pearson Education, 2016.
- 10 Pressman R. Software Engineering: A Practitioner's Approach. McGraw-Hill Education, 2014.
- 11 Elmasri R., Navathe S. Fundamentals of Database Systems. Pearson, 2016.
- 12 Laudon K., Laudon J. Management Information Systems. Pearson, 2020.
- 13 Freeman A. Pro Django. Apress, 2022.
- 14 Nielsen J. Usability Engineering. Morgan Kaufmann, 1994.
- 15 Welling L., Thomson L. PHP and MySQL Web Development. Addison-Wesley, 2016.

A.1 charity_project/urls.py файлы

```
from django.contrib import admin
from django.urls import path
from platforma import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.home_view, name='home'),
    path('projects/', views.project_list_view, name='project_list'),
    path('projects/<int:pk>/', views.project_detail_view, name='project_detail'),
    path('request-help/', views.request_help_view, name='request_help'),
    path('contribute/', views.contribute_view, name='contribute'),
    path('dashboard/', views.dashboard_view, name='dashboard'),
    path('about/', views.about_view, name='about'),
]
```

A.2 settings.py файлындағы негізгі баптаулар

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'platforma.apps.PlatformaConfig',
]
```

```
LANGUAGE_CODE = 'kk'
TIME_ZONE = 'Asia/Qyzylorda'
```

```
USE_I18N = True
USE_TZ = True
```

```
STATIC_URL = 'static/'
```

A.3 platforma/apps.py файлы

```
from django.apps import AppConfig
```

```
class PlatformaConfig(AppConfig):
    default_auto_field = 'django.db.models.AutoField'
    name = 'platforma'
    verbose_name = 'Қайырымдылық платформасы'
```

A.4 platforma/models.py файлы

```
from decimal import Decimal
```

```

from django.db import models
from django.urls import reverse
from django.utils import timezone
from django.utils.text import slugify

class TimeStampedModel(models.Model):
    created_at = models.DateTimeField(auto_now_add=True, verbose_name="Құрылған уақыты")
    updated_at = models.DateTimeField(auto_now=True, verbose_name="Жаңартылған уақыты")

    class Meta:
        abstract = True

class CharityProject(TimeStampedModel):
    STATUS_CHOICES = [
        ('draft', 'Дайындалуда'),
        ('active', 'Белсенді жинақ'),
        ('paused', 'Уақытша тоқтатылған'),
        ('completed', 'Аяқталған жоба'),
    ]

    CATEGORY_CHOICES = [
        ('food', 'Азық-түлік'),
        ('health', 'Денсаулық'),
        ('education', 'Білім'),
        ('clothes', 'Киім'),
        ('housing', 'Тұрғын үй'),
        ('other', 'Басқа'),
    ]

    URGENCY_CHOICES = [
        ('low', 'Төмен'),
        ('medium', 'Орташа'),
        ('high', 'Жоғары'),
        ('critical', 'Шұғыл'),
    ]

    title = models.CharField(max_length=200, verbose_name="Тақырыбы")
    slug = models.SlugField(max_length=220, unique=True, blank=True, verbose_name="URL атауы")
    description = models.TextField(verbose_name="Қысқаша сипаттамасы")
    story = models.TextField(blank=True, verbose_name="Толық оқиға")
    category = models.CharField(max_length=20, choices=CATEGORY_CHOICES, default='other', verbose_name="Категория")
    city = models.CharField(max_length=100, blank=True, verbose_name="Қала/аймақ")
    urgency = models.CharField(max_length=20, choices=URGENCY_CHOICES, default='medium', verbose_name="Маңыздылық")

```

```

target_amount = models.DecimalField(max_digits=12, decimal_places=0, default=0,
verbose_name="Қажет сома")
collected_amount = models.DecimalField(max_digits=12, decimal_places=0, default=0,
verbose_name="Жиналған сома")
deadline = models.DateField(null=True, blank=True, verbose_name="Мерзімі")
contact_phone = models.CharField(max_length=20, blank=True, verbose_name="Байланыс
телефоны")
contact_person = models.CharField(max_length=100, blank=True,
verbose_name="Жауапты адам")
status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='active',
verbose_name="Статус")
is_featured = models.BooleanField(default=False, verbose_name="Басты бетке шығару")
view_count = models.PositiveIntegerField(default=0, verbose_name="Қаралым саны")

```

```

class Meta:
    ordering = ['-is_featured', '-created_at']
    indexes = [
        models.Index(fields=['status', 'category']),
        models.Index(fields=['city']),
        models.Index(fields=['urgency']),
    ]
    verbose_name = "Қайырымдылық жобасы"
    verbose_name_plural = "Қайырымдылық жобалары"

```

```

@property
def progress_percent(self):
    if not self.target_amount:
        return 0
    percent = (self.collected_amount / self.target_amount) * Decimal('100')
    return min(100, int(percent))

```

```

@property
def remaining_amount(self):
    return max(Decimal('0'), self.target_amount - self.collected_amount)

```

```

@property
def days_left(self):
    if not self.deadline:
        return None
    return (self.deadline - timezone.localdate()).days

```

```

def get_absolute_url(self):
    return reverse('project_detail', kwargs={'pk': self.pk})

```

```

def save(self, *args, **kwargs):
    if not self.slug:
        base_slug = slugify(self.title, allow_unicode=True) or 'project'
        slug = base_slug
        counter = 2
        while CharityProject.objects.filter(slug=slug).exclude(pk=self.pk).exists():
            slug = f'{base_slug}-{counter}'

```

```

        counter += 1
        self.slug = slug
        super().save(*args, **kwargs)

    def __str__(self):
        return self.title

class ProjectUpdate(TimeStampedModel):
    project = models.ForeignKey(
        CharityProject,
        on_delete=models.CASCADE,
        related_name='updates',
        verbose_name="Жоба",
    )
    title = models.CharField(max_length=180, verbose_name="Жаңалық тақырыбы")
    body = models.TextField(verbose_name="Жаңалық мәтіні")
    is_public = models.BooleanField(default=True, verbose_name="Сайтта көрсету")

    class Meta:
        ordering = ['-created_at']
        verbose_name = "Жоба жаңалығы"
        verbose_name_plural = "Жоба жаңалықтары"

    def __str__(self):
        return f"{self.project.title}: {self.title}"

class HelpRequest(TimeStampedModel):
    REQUEST_TYPE_CHOICES = [
        ('food', 'Азық-түлік'),
        ('health', 'Денсаулық'),
        ('education', 'Білім'),
        ('clothes', 'Киім'),
        ('housing', 'Тұрғын үй'),
        ('other', 'Басқа'),
    ]

    STATUS_CHOICES = [
        ('new', 'Жаңа'),
        ('review', 'Тексерілуде'),
        ('accepted', 'Қабылданды'),
        ('rejected', 'Қабылданбады'),
        ('closed', 'Жабылды'),
    ]

    PRIORITY_CHOICES = [
        ('normal', 'Қалыпты'),
        ('high', 'Маңызды'),
        ('urgent', 'Шұғыл'),
    ]

```

```

name = models.CharField(max_length=100, verbose_name="Аты-жөні")
phone = models.CharField(max_length=20, verbose_name="Телефон")
email = models.EmailField(blank=True, verbose_name="Email")
city = models.CharField(max_length=100, blank=True, verbose_name="Қала/аймақ")
request_type = models.CharField(max_length=20, choices=REQUEST_TYPE_CHOICES,
default='other', verbose_name="Көмек түрі")
priority = models.CharField(max_length=20, choices=PRIORITY_CHOICES,
default='normal', verbose_name="Басымдық")
message = models.TextField(verbose_name="Қандай көмек керек?")
status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='new',
verbose_name="Статус")
moderator_note = models.TextField(blank=True, verbose_name="Модератор жазбасы")

class Meta:
    ordering = ['-created_at']
    indexes = [
        models.Index(fields=['status', 'priority']),
        models.Index(fields=['request_type']),
        models.Index(fields=['city']),
    ]
    verbose_name = "Көмек өтініші"
    verbose_name_plural = "Көмек өтініштері"

def __str__(self):
    return f"Өтініш: {self.name} - {self.phone}"

class Contribution(TimeStampedModel):
    METHOD_CHOICES = [
        ('money', 'Қаржылай көмек'),
        ('goods', 'Заттай көмек'),
        ('volunteer', 'Ерікті болу'),
        ('consulting', 'Кеңес беру'),
    ]

    STATUS_CHOICES = [
        ('new', 'Жаңа'),
        ('confirmed', 'Расталды'),
        ('cancelled', 'Болдырылмады'),
    ]

    project = models.ForeignKey(
        CharityProject,
        on_delete=models.CASCADE,
        related_name='contributions',
        verbose_name="Жоба",
    )
    name = models.CharField(max_length=100, verbose_name="Аты-жөні")
    phone = models.CharField(max_length=20, verbose_name="Телефон")
    email = models.EmailField(blank=True, verbose_name="Email")

```

```

method = models.CharField(max_length=20, choices=METHOD_CHOICES,
default='money', verbose_name="Көмек түрі")
amount = models.DecimalField(max_digits=12, decimal_places=0, default=0,
verbose_name="Сума")
message = models.TextField(blank=True, verbose_name="Қосымша хабарлама")
status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='new',
verbose_name="Статус")
is_anonymous = models.BooleanField(default=False, verbose_name="Атымды көрсетпей")
applied_to_total = models.BooleanField(default=False, verbose_name="Жоба сомасына
қосылды")

```

```

class Meta:
    ordering = ['-created_at']
    indexes = [
        models.Index(fields=['status', 'method']),
        models.Index(fields=['project', 'created_at']),
    ]
    verbose_name = "Үлес қосу өтініші"
    verbose_name_plural = "Үлес қосу өтініштері"

```

```

def __str__(self):
    return f"{self.name} - {self.project.title}"

```

A.5 platforma/forms.py файлы

```

from django import forms

```

```

from .models import Contribution, HelpRequest

```

```

class HelpRequestForm(forms.ModelForm):
    class Meta:
        model = HelpRequest
        fields = ['name', 'phone', 'email', 'city', 'request_type', 'priority', 'message']
        labels = {
            'name': 'Аты-жөніңіз',
            'phone': 'Телефон нөмірі',
            'email': 'Email',
            'city': 'Қала/аймақ',
            'request_type': 'Көмек түрі',
            'priority': 'Маңыздылығы',
            'message': 'Жағдайды қысқаша сипаттаңыз',
        }
        widgets = {
            'name': forms.TextInput(attrs={'placeholder': 'Мысалы: Айдана Қасым'}),
            'phone': forms.TextInput(attrs={'placeholder': '+7 700 000 00 00'}),
            'email': forms.EmailInput(attrs={'placeholder': 'name@example.com'}),
            'city': forms.TextInput(attrs={'placeholder': 'Алматы, Астана, Шымкент...'}),
            'message': forms.Textarea(attrs={'placeholder': 'Қандай көмек қажет екенін жазыңыз',
'rows': 5}),
        }

```



```

class ContributionForm(forms.ModelForm):
    class Meta:
        model = Contribution
        fields = ['project', 'name', 'phone', 'email', 'method', 'amount', 'is_anonymous', 'message']
        labels = {
            'project': 'Жоба',
            'name': 'АТЫ-ЖӨНІҢІЗ',
            'phone': 'Телефон нөмірі',
            'email': 'Email',
            'method': 'Көмек түрі',
            'amount': 'Сума',
            'is_anonymous': 'Атымды көрсетпеу',
            'message': 'Қосымша хабарлама',
        }
        widgets = {
            'name': forms.TextInput(attrs={'placeholder': 'Мысалы: Ерлан Нұрлан'}),
            'phone': forms.TextInput(attrs={'placeholder': '+7 701 000 00 00'}),
            'email': forms.EmailInput(attrs={'placeholder': 'name@example.com'}),
            'amount': forms.NumberInput(attrs={'min': 0, 'placeholder': '10000'}),
            'message': forms.Textarea(attrs={'placeholder': 'Қалай көмектесе алатыныңызды жазыңыз', 'rows': 4}),
        }

    def __init__(self, *args, **kwargs):
        active_projects = kwargs.pop('active_projects', None)
        selected_project = kwargs.pop('selected_project', None)
        super().__init__(*args, **kwargs)

        if active_projects is not None:
            self.fields['project'].queryset = active_projects

        if selected_project is not None:
            self.fields['project'].initial = selected_project

        self.fields['project'].empty_label = 'Жобаны таңдаңыз'

```

A.6 platforma/views.py файлы

```

from decimal import Decimal

from django.contrib import messages
from django.db.models import Count, Q, Sum
from django.shortcuts import get_object_or_404, redirect, render

from .forms import ContributionForm, HelpRequestForm
from .models import CharityProject, Contribution, HelpRequest, ProjectUpdate

def get_active_projects():
    return CharityProject.objects.filter(status='active')

```

```

def get_platform_stats():
    active_projects = get_active_projects()
    total_target = active_projects.aggregate(total=Sum('target_amount'))['total'] or Decimal('0')
    total_collected = active_projects.aggregate(total=Sum('collected_amount'))['total'] or
    Decimal('0')

    return {
        'active_count': active_projects.count(),
        'completed_count': CharityProject.objects.filter(status='completed').count(),
        'request_count': HelpRequest.objects.count(),
        'contribution_count': Contribution.objects.count(),
        'total_target': total_target,
        'total_collected': total_collected,
    }

def apply_contribution_to_project(contribution):
    if contribution.method == 'money' and contribution.amount > 0 and not
    contribution.applied_to_total:
        project = contribution.project
        project.collected_amount += contribution.amount
        project.save(update_fields=['collected_amount', 'updated_at'])

        contribution.applied_to_total = True
        contribution.save(update_fields=['applied_to_total', 'updated_at'])

def home_view(request):
    featured_projects = get_active_projects().filter(is_featured=True)[:3]

    if not featured_projects:
        featured_projects = get_active_projects()[:3]

    context = {
        **get_platform_stats(),
        'featured_projects': featured_projects,
        'latest_updates': ProjectUpdate.objects.filter(is_public=True).select_related('project')[:4],
        'latest_completed': CharityProject.objects.filter(status='completed')[:3],
    }

    return render(request, 'platforma/home.html', context)

def project_list_view(request):
    projects = CharityProject.objects.all()

    query = request.GET.get('q', "").strip()
    category = request.GET.get('category', "").strip()
    city = request.GET.get('city', "").strip()
    urgency = request.GET.get('urgency', "").strip()

```

```

status = request.GET.get('status', 'active').strip()

if status:
    projects = projects.filter(status=status)

if query:
    projects = projects.filter(
        Q(title__icontains=query) |
        Q(description__icontains=query) |
        Q(story__icontains=query) |
        Q(contact_phone__icontains=query)
    )

if category:
    projects = projects.filter(category=category)

if city:
    projects = projects.filter(city__icontains=city)

if urgency:
    projects = projects.filter(urgency=urgency)

context = {
    **get_platform_stats(),
    'projects': projects,
    'categories': CharityProject.CATEGORY_CHOICES,
    'urgencies': CharityProject.URGENCY_CHOICES,
    'statuses': CharityProject.STATUS_CHOICES,
    'query': query,
    'selected_category': category,
    'selected_city': city,
    'selected_urgency': urgency,
    'selected_status': status,
}

return render(request, 'platforma/project_list.html', context)

def project_detail_view(request, pk):
    project = get_object_or_404(CharityProject, pk=pk)
    project.view_count += 1
    project.save(update_fields=['view_count'])

    contribution_form = ContributionForm(
        active_projects=get_active_projects(),
        selected_project=project,
    )

    if request.method == 'POST':
        contribution_form = ContributionForm(
            request.POST,

```

```

        active_projects=get_active_projects(),
        selected_project=project,
    )

    if contribution_form.is_valid():
        contribution = contribution_form.save()
        apply_contribution_to_project(contribution)
        messages.success(request, 'Үлес қосу өтінішіңіз қабылданды. Модератор сізбен
байланысады.')
        return redirect(project.get_absolute_url())

    context = {
        **get_platform_stats(),
        'project': project,
        'updates': project.updates.filter(is_public=True),
        'contributions': project.contributions.filter(status__in=['new', 'confirmed'][:8],
        'contribution_form': contribution_form,
    }

    return render(request, 'platforma/project_detail.html', context)

def request_help_view(request):
    help_form = HelpRequestForm()

    if request.method == 'POST':
        help_form = HelpRequestForm(request.POST)

        if help_form.is_valid():
            help_form.save()
            messages.success(request, 'Өтінішіңіз қабылданды. Модератор жақын арада
байланысады.')
            return redirect('request_help')

    context = {
        **get_platform_stats(),
        'help_form': help_form,
        'recent_requests': HelpRequest.objects.exclude(status='rejected')[:6],
    }

    return render(request, 'platforma/request_help.html', context)

def contribute_view(request):
    contribution_form = ContributionForm(active_projects=get_active_projects())

    if request.method == 'POST':
        contribution_form = ContributionForm(
            request.POST,
            active_projects=get_active_projects(),
        )

```

```

        if contribution_form.is_valid():
            contribution = contribution_form.save()
            apply_contribution_to_project(contribution)
            messages.success(request, 'Үлес қосу өтінішіңіз қабылданды. Рахмет!')
            return redirect('contribute')

    context = {
        **get_platform_stats(),
        'contribution_form': contribution_form,
        'active_projects': get_active_projects()[:6],
    }

    return render(request, 'platforma/contribute.html', context)

def dashboard_view(request):
    projects_by_category = CharityProject.objects.values('category').annotate(count=Count('id')).order_by('category')
    requests_by_status = HelpRequest.objects.values('status').annotate(count=Count('id')).order_by('status')
    contributions_by_method = Contribution.objects.values('method').annotate(
        count=Count('id'),
        total=Sum('amount'),
    ).order_by('method')

    context = {
        **get_platform_stats(),
        'projects_by_category': projects_by_category,
        'requests_by_status': requests_by_status,
        'contributions_by_method': contributions_by_method,
        'latest_requests': HelpRequest.objects.all()[:8],
        'latest_contributions': Contribution.objects.select_related('project').all()[:8],
        'top_projects': get_active_projects().order_by('-collected_amount')[:5],
    }

    return render(request, 'platforma/dashboard.html', context)

def about_view(request):
    return render(request, 'platforma/about.html', get_platform_stats())

```

A.7 platforma/admin.py файлы

```

from django.contrib import admin

from .models import CharityProject, Contribution, HelpRequest, ProjectUpdate

admin.site.site_header = 'Мейірім Pro басқару панелі'
admin.site.site_title = 'Мейірім Pro'
admin.site.index_title = 'Қайырымдылық платформасын басқару'

```

```

class ProjectUpdateInline(admin.TabularInline):
    model = ProjectUpdate
    extra = 0
    fields = ('title', 'is_public', 'created_at')
    readonly_fields = ('created_at',)
    verbose_name = 'Жоба жаңалығы'
    verbose_name_plural = 'Жоба жаңалықтары'

@admin.register(CharityProject)
class CharityProjectAdmin(admin.ModelAdmin):
    list_display = (
        'title',
        'category_label',
        'city',
        'urgency_label',
        'status_label',
        'target_amount',
        'collected_amount',
        'is_featured',
        'created_at',
    )
    list_filter = ('status', 'category', 'urgency', 'is_featured', 'city')
    search_fields = ('title', 'description', 'story', 'contact_phone', 'contact_person')
    prepopulated_fields = {'slug': ('title',)}
    readonly_fields = ('view_count', 'created_at', 'updated_at')
    ordering = ('-is_featured', '-created_at')
    inlines = [ProjectUpdateInline]

    fieldsets = (
        ('Негізгі ақпарат', {
            'fields': ('title', 'slug', 'description', 'story', 'status', 'is_featured')
        }),
        ('Санат және орналасу', {
            'fields': ('category', 'city', 'urgency', 'deadline')
        }),
        ('Қаржылық ақпарат', {
            'fields': ('target_amount', 'collected_amount')
        }),
        ('Байланыс', {
            'fields': ('contact_person', 'contact_phone')
        }),
        ('Жүйелік ақпарат', {
            'fields': ('view_count', 'created_at', 'updated_at')
        }),
    )

@admin.display(description='Категория')
def category_label(self, obj):

```

```

        return obj.get_category_display()

@admin.display(description='Маңыздылық')
def urgency_label(self, obj):
    return obj.get_urgency_display()

@admin.display(description='Статус')
def status_label(self, obj):
    return obj.get_status_display()

@admin.register(ProjectUpdate)
class ProjectUpdateAdmin(admin.ModelAdmin):
    list_display = ('title', 'project', 'is_public', 'created_at')
    list_filter = ('is_public', 'project')
    search_fields = ('title', 'body', 'project__title')
    ordering = ('-created_at',)

@admin.register(HelpRequest)
class HelpRequestAdmin(admin.ModelAdmin):
    list_display = (
        'name',
        'phone',
        'city',
        'request_type_label',
        'priority_label',
        'status_label',
        'created_at',
    )
    list_filter = ('status', 'priority', 'request_type', 'city')
    search_fields = ('name', 'phone', 'email', 'city', 'message')
    readonly_fields = ('created_at', 'updated_at')
    ordering = ('-created_at',)

@admin.display(description='Көмек түрі')
def request_type_label(self, obj):
    return obj.get_request_type_display()

@admin.display(description='Басымдық')
def priority_label(self, obj):
    return obj.get_priority_display()

@admin.display(description='Статус')
def status_label(self, obj):
    return obj.get_status_display()

@admin.register(Contribution)
class ContributionAdmin(admin.ModelAdmin):
    list_display = (

```

```

        'name',
        'phone',
        'project',
        'method_label',
        'amount',
        'status_label',
        'applied_to_total',
        'created_at',
    )
    list_filter = ('status', 'method', 'applied_to_total', 'project')
    search_fields = ('name', 'phone', 'email', 'message', 'project__title')
    readonly_fields = ('created_at', 'updated_at')
    ordering = ('-created_at',)

@admin.display(description='Көмек түрі')
def method_label(self, obj):
    return obj.get_method_display()

@admin.display(description='Статус')
def status_label(self, obj):
    return obj.get_status_display()

```

A.8 seed_demo.py демо деректер командасының негізгі бөлігі

```

from datetime import timedelta
from decimal import Decimal

```

```

from django.core.management.base import BaseCommand
from django.utils import timezone

```

```

from platforma.models import CharityProject, Contribution, HelpRequest, ProjectUpdate

```

```

class Command(BaseCommand):
    help = 'Мейірім Pro платформасын демо деректермен толтырады.'

    def handle(self, *args, **options):
        today = timezone.localdate()

        projects = [
            {
                'title': 'Алматыдағы көпбалалы отбасыға қысқы киім',
                'defaults': {
                    'description': '5 балалы отбасыға қысқы киім, аяқ киім және жылы көрпе қажет.',
                    'story': 'Отбасы уақытша жалдамалы пәтерде тұрады. Қыс мезгілі жақындағандықтан балаларға жылы киім жинау маңызды.',
                    'category': 'clothes',
                    'city': 'Алматы',
                    'urgency': 'high',
                    'target_amount': Decimal('450000'),
                    'collected_amount': Decimal('285000'),
                }
            }
        ]

```



```

        'deadline': today + timedelta(days=18),
        'contact_phone': '+7 701 214 45 18',
        'contact_person': 'Модератор Айгүл',
        'status': 'active',
        'is_featured': True,
    },
},
{
    'title': 'ШЫМКЕНТТЕ ем қабылдап жатқан балаға дәрі-дәрмек',
    'defaults': {
        'description': 'Ұзақ ем қабылдап жатқан балаға дәрі-дәрмек пен талдау
шығындарына көмек керек.',
        'story': 'Жиналған қаражат дәрілерге, анализдерге және жол шығынына
жұмсалады.',
        'category': 'health',
        'city': 'ШЫМКЕНТ',
        'urgency': 'critical',
        'target_amount': Decimal('850000'),
        'collected_amount': Decimal('512000'),
        'deadline': today + timedelta(days=10),
        'contact_phone': '+7 707 500 22 11',
        'contact_person': 'Ерікті Данияр',
        'status': 'active',
        'is_featured': True,
    },
},
{
    'title': 'Ауыл мектебіне оқу құралдары',
    'defaults': {
        'description': 'Шағын мектепке дәптер, қалам, сөмке және кітаптар қажет.',
        'story': 'Жоба оқу жылына дайындықты жеңілдетуге бағытталған.',
        'category': 'education',
        'city': 'Түркістан облысы',
        'urgency': 'medium',
        'target_amount': Decimal('620000'),
        'collected_amount': Decimal('620000'),
        'deadline': today - timedelta(days=5),
        'contact_phone': '+7 778 808 70 60',
        'contact_person': 'Мектеп үйлестірушісі',
        'status': 'completed',
        'is_featured': False,
    },
},
]

```

```
created_projects = {}
```

```
for item in projects:
```

```

    project, _ = CharityProject.objects.update_or_create(
        title=item['title'],
        defaults=item['defaults'],

```

```

    )
    created_projects[item['title']] = project

    ProjectUpdate.objects.update_or_create(
        project=created_projects['Алматыдағы көпбалалы отбасыға қысқы киім'],
        title='Алғашқы жеткізу жасалды',
        defaults={
            'body': 'Балаларға арналған күртешелер мен аяқ киімдердің бір бөлігі
жеткізілді.',
            'is_public': True,
        },
    )

    HelpRequest.objects.update_or_create(
        name='Гүлнұр Сәрсен',
        phone='+7 700 900 10 20',
        defaults={
            'email': 'gulnur@example.com',
            'city': 'Тараз',
            'request_type': 'food',
            'priority': 'urgent',
            'message': 'Үш баламен уақытша қиын жағдайда қалдық, азық-түлік көмегі
қажет.',
            'status': 'review',
        },
    )

    Contribution.objects.update_or_create(
        project=created_projects['Шымкентте ем қабылдап жатқан балаға дәрі-дәрмек'],
        name='Аноним',
        phone='+7 700 000 00 01',
        defaults={
            'email': '',
            'method': 'money',
            'amount': Decimal('120000'),
            'message': '',
            'status': 'confirmed',
            'is_anonymous': True,
            'applied_to_total': True,
        },
    )

```

```

self.stdout.write(self.style.SUCCESS('Demo data loaded successfully.))

```

A.9 platforma/tests.py файлы

```

from django.test import TestCase, override_settings
from django.urls import reverse

```

```

from .models import CharityProject, Contribution, HelpRequest, ProjectUpdate

```

```

@override_settings(ALLOWED_HOSTS=['testserver', '127.0.0.1'])
class PlatformPageTests(TestCase):
    def setUp(self):
        self.project = CharityProject.objects.create(
            title='Жылы киім жинау',
            description='Қыс мезгіліне арналған көмек.',
            story='Отбасыға жылы киім мен аяқ киім қажет.',
            category='clothes',
            city='Алматы',
            urgency='high',
            target_amount=150000,
            collected_amount=45000,
            contact_phone='+7 700 111 22 33',
            status='active',
            is_featured=True,
        )

        CharityProject.objects.create(
            title='Оқу құралдары',
            description='Балаларға оқу құралдары берілді.',
            category='education',
            city='Астана',
            status='completed',
        )

        ProjectUpdate.objects.create(
            project=self.project,
            title='Алғашқы жеткізу жасалды',
            body='Киімдердің бір бөлігі отбасыға берілді.',
        )

    def test_main_pages_render(self):
        urls = [
            reverse('home'),
            reverse('project_list'),
            self.project.get_absolute_url(),
            reverse('request_help'),
            reverse('contribute'),
            reverse('dashboard'),
            reverse('about'),
        ]

        for url in urls:
            response = self.client.get(url)
            self.assertEqual(response.status_code, 200)
            self.assertContains(response, 'Мейірім Pro')

    def test_project_list_filters(self):
        response = self.client.get(
            reverse('project_list'),
            {

```

```

        'q': 'киім',
        'category': 'clothes',
        'city': 'Алматы',
        'urgency': 'high',
        'status': 'active',
    },
)

self.assertEqual(response.status_code, 200)
self.assertContains(response, 'Жылы киім жинау')
self.assertNotContains(response, 'Оқу құралдары')

def test_help_request_form_saves_request(self):
    response = self.client.post(
        reverse('request_help'),
        {
            'name': 'Айдана Қасым',
            'phone': '+7 700 000 00 00',
            'email': 'aidana@example.com',
            'city': 'Шымкент',
            'request_type': 'food',
            'priority': 'urgent',
            'message': 'Азық-түлік көмегі қажет.',
        },
        follow=True,
    )

    self.assertEqual(response.status_code, 200)
    request = HelpRequest.objects.get(name='Айдана Қасым')
    self.assertEqual(request.status, 'new')
    self.assertEqual(request.priority, 'urgent')
    self.assertContains(response, 'Өтінішіңіз қабылданды')

def test_contribution_form_updates_project_total(self):
    response = self.client.post(
        reverse('contribute'),
        {
            'project': self.project.id,
            'name': 'Ерлан Нұрлан',
            'phone': '+7 701 000 00 00',
            'email': 'erlan@example.com',
            'method': 'money',
            'amount': 50000,
            'message': 'Бүгін аударамын.',
        },
        follow=True,
    )

    self.project.refresh_from_db()

    self.assertEqual(response.status_code, 200)

```

```

self.assertTrue(
    Contribution.objects.filter(
        name='Ерлан Нұрлан',
        project=self.project,
    ).exists()
)
self.assertEqual(self.project.collected_amount, 95000)
self.assertContains(response, 'Үлес қосу өтінішіңіз қабылданды')

```

A.10 Негізгі HTML шаблон құрылымы

```

<!-- templates/platforma/base.html -->

<!DOCTYPE html>
<html lang="kk">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Мейірім Pro{% endblock %}</title>

    <link          href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link          href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.2/css/all.min.css"
rel="stylesheet">
</head>
<body>
    <nav class="topbar py-3">
        <div class="container d-flex justify-content-between align-items-center">
            <a href="{% url 'home' %}" class="text-decoration-none text-dark">
                <strong>Мейірім Pro</strong>
            </a>

            <div class="d-flex gap-3">
                <a href="{% url 'project_list' %}">Жобалар</a>
                <a href="{% url 'request_help' %}">Көмек сұрай</a>
                <a href="{% url 'contribute' %}">Үлес қосу</a>
                <a href="{% url 'dashboard' %}">Dashboard</a>
                <a href="{% url 'about' %}">Платформа</a>
                <a href="/admin/">Admin</a>
            </div>
        </div>
    </nav>

    {% if messages %}
        <div class="container mt-4">
            {% for message in messages %}
                <div class="alert alert-success">
                    {{ message }}
                </div>
            {% endfor %}
        </div>
    {% endif %}

```

```

{% block content %} {% endblock %}

<footer class="border-top bg-white py-4 mt-5">
  <div class="container">
    <strong>Мейірім Pro</strong>
    <span class="text-secondary">2026 - дипломдық жоба</span>
  </div>
</footer>
</body>
</html>
<!-- templates/platforma/home.html -->

{% extends "platforma/base.html" %}

{% block title %}Мейірім Pro - Басты бет{% endblock %}

{% block content %}
<header class="hero">
  <div class="container py-5">
    <h1>Қайырымдылық жобаларын басқару платформасы</h1>
    <p>
      Жобалар, өтініштер, үлестер, жаңалықтар және аналитика
      бір жүйеде жұмыс істейді.
    </p>

    <a href="{% url 'project_list' %}" class="btn btn-primary">Жобаларды көру</a>
    <a href="{% url 'request_help' %}" class="btn btn-light">Көмек сұрау</a>
  </div>
</header>

<main class="container my-5">
  <h2>Ұсынылған жобалар</h2>

  <div class="row">
    {% for project in featured_projects %}
      <div class="col-md-4">
        <div class="card p-3">
          <h3>{{ project.title }}</h3>
          <p>{{ project.description }}</p>
          <p>{{ project.collected_amount }} ₸ / {{ project.target_amount }} ₸</p>

          <div class="progress">
            <div class="progress-bar" style="width: {{ project.progress_percent }}%></div>
          </div>

          <a href="{{ project.get_absolute_url }}" class="btn btn-success mt-3">
            Толығырақ
          </a>
        </div>
      </div>
    {% endfor %}
  </div>

```

```

        </div>
        {% empty %}
        <p>Әзірге белсенді жоба жоқ.</p>
        {% endfor %}
    </div>
</main>
{% endblock %}
<!-- templates/platforma/project_list.html -->

{% extends "platforma/base.html" %}

{% block content %}
<main class="container my-5">
    <h1>Қайырымдылық жобалары</h1>

    <form method="get" class="row g-2 mb-4">
        <div class="col-md-3">
            <input type="search" name="q" value="{{ query }}" placeholder="Іздеу">
        </div>

        <div class="col-md-3">
            <input type="text" name="city" value="{{ selected_city }}" placeholder="Қала">
        </div>

        <div class="col-md-3">
            <select name="category">
                <option value="">Барлық категория</option>
                {% for value, label in categories %}
                <option value="{{ value }}" {% if selected_category == value %}selected{%
endif %}>
                    {{ label }}
                </option>
                {% endfor %}
            </select>
        </div>

        <div class="col-md-3">
            <button type="submit" class="btn btn-primary w-100">Іздеу</button>
        </div>
    </form>

    <div class="row">
        {% for project in projects %}
        <div class="col-md-4">
            <div class="card p-3 mb-3">
                <h3>{{ project.title }}</h3>
                <p>{{ project.description }}</p>
                <p>{{ project.get_category_display }} | {{ project.city }}</p>
                <a href="{{ project.get_absolute_url }}" class="btn btn-success">
                    Толығырақ
                </a>
            </div>
        </div>
        </div>
    </div>

```

```

        </div>
    </div>
    {% empty %}
        <p>Сәйкес жоба табылмады.</p>
    {% endfor %}
</div>
</main>
{% endblock %}
<!-- templates/platforma/project_detail.html -->

{% extends "platforma/base.html" %}

{% block content %}
<main class="container my-5">
    <h1>{{ project.title }}</h1>

    <p>{{ project.description }}</p>
    <p>{{ project.story }}</p>

    <p>Категория: {{ project.get_category_display }}</p>
    <p>Қала: {{ project.city }}</p>
    <p>Мәртебе: {{ project.get_status_display }}</p>

    <h3>Қаржылық прогресс</h3>
    <p>{{ project.collected_amount }} ₸ / {{ project.target_amount }} ₸</p>

    <div class="progress mb-3">
        <div class="progress-bar" style="width: {{ project.progress_percent }}%></div>
    </div>

    <h3>Осы жобаға үлес қосу</h3>

    <form method="post">
        {% csrf_token %}
        {{ contribution_form.as_p }}
        <button type="submit" class="btn btn-success">Үлес қосу</button>
    </form>

    <h3 class="mt-4">Жоба жаңалықтары</h3>

    {% for update in updates %}
        <div class="border-bottom mb-3">
            <strong>{{ update.title }}</strong>
            <p>{{ update.body }}</p>
        </div>
    {% empty %}
        <p>Бұл жоба бойынша жаңалық жоқ.</p>
    {% endfor %}
</main>
{% endblock %}
<!-- templates/platforma/request_help.html -->

```



```

{% extends "platforma/base.html" %}

{% block content %}
<main class="container my-5">
  <h1>Көмекке өтініш қалдыру</h1>

  <form method="post">
    {% csrf_token %}
    {{ help_form.as_p }}
    <button type="submit" class="btn btn-primary">Өтініш жіберу</button>
  </form>

  <h2 class="mt-5">Соңғы өтініштер</h2>

  <table class="table">
    <tr>
      <th>АТЫ</th>
      <th>Көмек түрі</th>
      <th>Статус</th>
    </tr>

    {% for item in recent_requests %}
      <tr>
        <td>{{ item.name }}</td>
        <td>{{ item.get_request_type_display }}</td>
        <td>{{ item.get_status_display }}</td>
      </tr>
    {% endfor %}
  </table>
</main>
{% endblock %}
<!-- templates/platforma/contribute.html -->

```

```

{% extends "platforma/base.html" %}

{% block content %}
<main class="container my-5">
  <h1>Үлес қосу</h1>

  <form method="post">
    {% csrf_token %}
    {{ contribution_form.as_p }}
    <button type="submit" class="btn btn-success">Ұсыну</button>
  </form>

  <h2 class="mt-5">Қолдауға болатын жобалар</h2>

  <div class="row">
    {% for project in active_projects %}
      <div class="col-md-4">

```

```

        <div class="card p-3 mb-3">
            <h3>{{ project.title }}</h3>
            <p>{{ project.description }}</p>
            <a href="{{ project.get_absolute_url }}" class="btn btn-primary">
                Толығырақ
            </a>
        </div>
    </div>
    {% endfor %}
</div>
</main>
{% endblock %}
<!-- templates/platforma/dashboard.html -->

{% extends "platforma/base.html" %}

{% block content %}
<main class="container my-5">
    <h1>Dashboard</h1>

    <div class="row">
        <div class="col-md-3">Белсенді жобалар: {{ active_count }}</div>
        <div class="col-md-3">Аяқталған жобалар: {{ completed_count }}</div>
        <div class="col-md-3">Өтініштер: {{ request_count }}</div>
        <div class="col-md-3">Үлестер: {{ contribution_count }}</div>
    </div>

    <h2 class="mt-5">Соңғы көмек өтініштері</h2>

    <table class="table">
        <tr>
            <th>Аты</th>
            <th>Қала</th>
            <th>Статус</th>
        </tr>

        {% for item in latest_requests %}
            <tr>
                <td>{{ item.name }}</td>
                <td>{{ item.city }}</td>
                <td>{{ item.get_status_display }}</td>
            </tr>
        {% endfor %}
    </table>

    <h2 class="mt-5">Соңғы үлестер</h2>

    <table class="table">
        <tr>
            <th>Аты</th>
            <th>Жоба</th>

```

```

        <th>Coma</th>
    </tr>

    {% for item in latest_contributions %}
        <tr>
            <td>{% if item.is_anonymous %}Аноним{% else %}{{ item.name }}{% endif
%}</td>
            <td>{{ item.project.title }}</td>
            <td>{{ item.amount }} ₮</td>
        </tr>
    {% endfor %}
</table>
</main>
{% endblock %}

```